INFORME PROYECTO UNITY

AUTOR: ENRIQUE CAÑADAS GONZÁLEZ

ANIMACIÓN DIGITAL
GRADO DE INGENIERÍA INFORMÁTICA



Contenido

TERRENO Y DISEÑO DE ESTE 3
Creación del terreno y uso de texturas y plantas 3
Skybox 7
MODELO Y ANIMACIONES DEL PERSONAJE PRINCIPAL 8
Animaciones del personaje principal 11
Animator "Paladin Controller" 12
Blend tree 12
Animación de salto 13
Animación de caída 14
Animación de impacto en el jugador 14
Animación de muerte en el personaje 15
Animación de celebración 16
Animación de ataque 17
Animación de defensa e impacto en el escudo 19
MODELO Y ANIMACIONES DEL ENEMIGO 21
Animaciones del enemigo 23
Animator "Enemigo controller" 23
Animación de andar del enemigo 24
Animación de perseguir al jugador 24
Animación de impacto en el enemigo 25
Animación de muerte del enemigo 26
Animación de ataque del enemigo 27
CÁMARA DEL JUEGO 28
ESCENA "MENÚ DE ENTRADA" 29
CANVAS DEL PERSONAJE Y PANELES DE VICTORIA Y DERROTA 32
Slider de vida del personaje y texto con el valor numérico 32
Panel con mensaje de misión 33
Panel de derrota 34
Panel de victoria 36
CANVAS DEL ENEMIGO 37
Slider de vida del enemigo 37
ITEMS VIDA 38
SONIDOS 39

ANOTACIÓN:

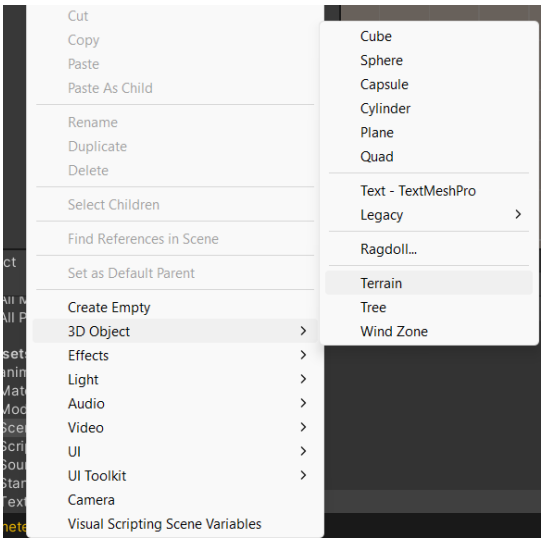
TODOS LOS MODELOS Y ANIMACIONES DE ESTE TRABAJO HAN SIDO SACADOS DE HTTPS://WWW.MIXAMO.COM/#/

TERRENO Y DISEÑO DE ESTE

En este apartado se pretende mostrar los pasos y el diseño final del terreno que usaremos.

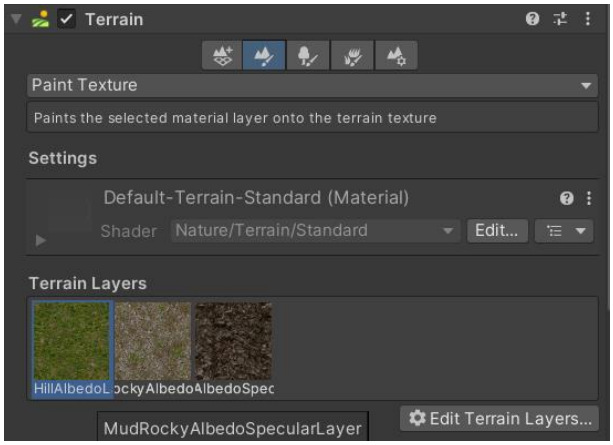
Creación del terreno y uso de texturas y plantas

En primer lugar, en la jerarquía, creo el terreno nuevo.

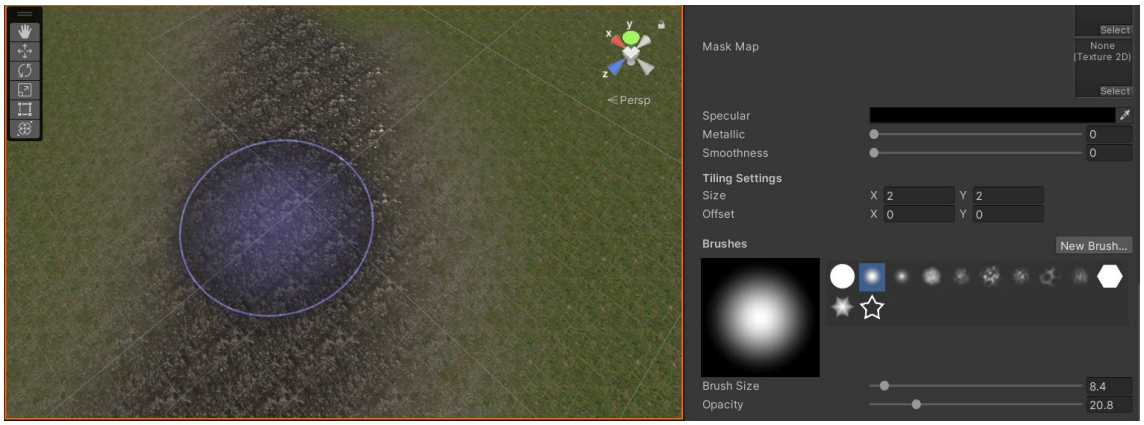


Una vez creado el terreno, importo las texturas que voy a utilizar para pintar el terreno. En mi caso he usado las texturas: MudRockyAlbedoSpecular, GrassHillAlbedo y GrassRockyAlbedo las cuales se encuentran en la carpeta StandardAssests->Enviroment->TerrainAssets->SurfaceTextures. Para esto, en el apartado de Terrain Layer, he creado los tres layers con esas tres texturas que he usado.

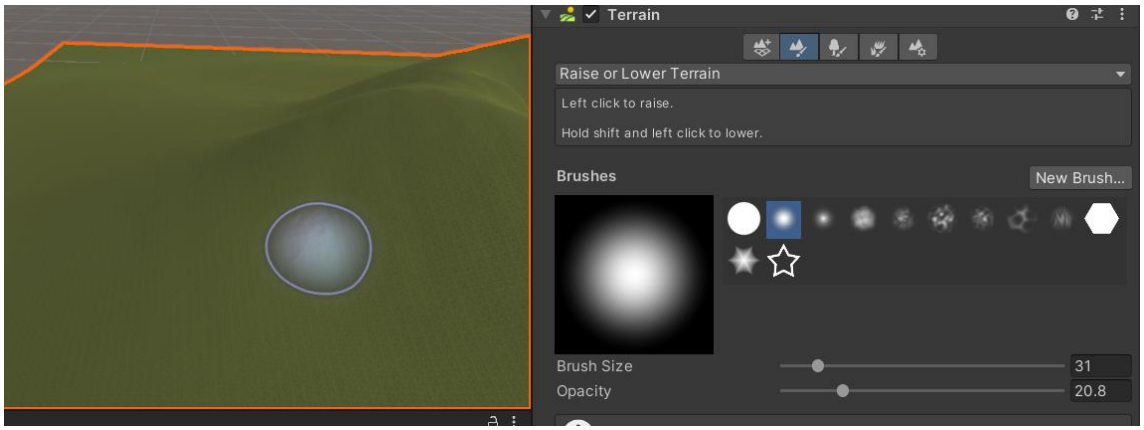
La que se expande por todo el terreno es la GrassHillAlbedo.



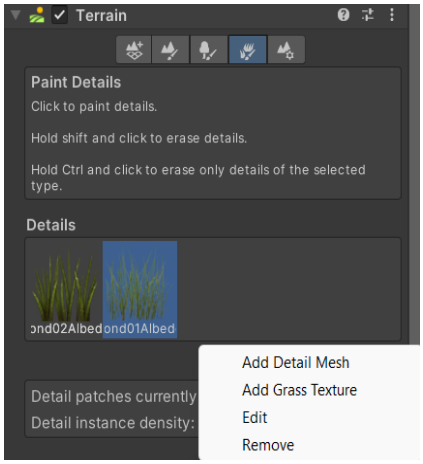
A continuación, en el modo Paint Texture, he ido seleccionando entre los tres layers que he creado, y jugando con el tamaño de la brocha y la opacidad he ido pintando esas texturas sobre el terreno.

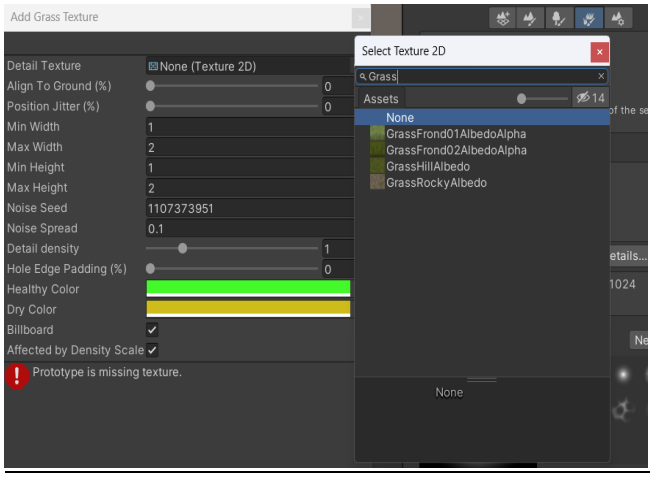


Una vez pintado el terreno con las texturas que hemos mencionado antes, he creado montañas en el terreno con la herramienta Raise or lower terrain, jugando con el Brush size y la opacity.

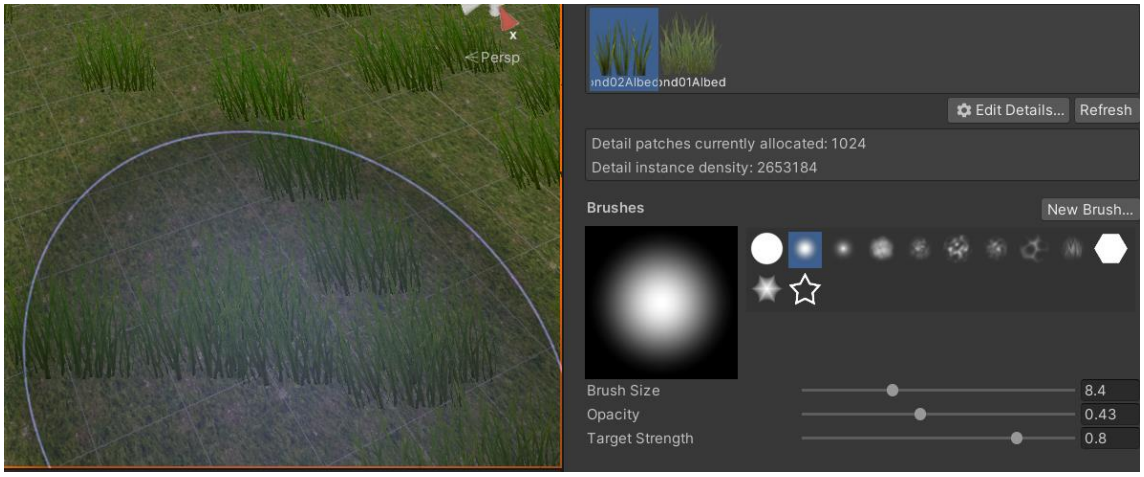


Tras esto, he decidio poner algunos detalles como hierba. Para ello he seleccionado la opción Paint Details->Edit Details->Add Grass Texture, añadiendo las texturas GrassFrond01AlbedoAlpha y GrassFrond02AlbedoAlpha de la carpeta StandardAssets->Enviroment->TerrainAssets->BillBoardTexture.

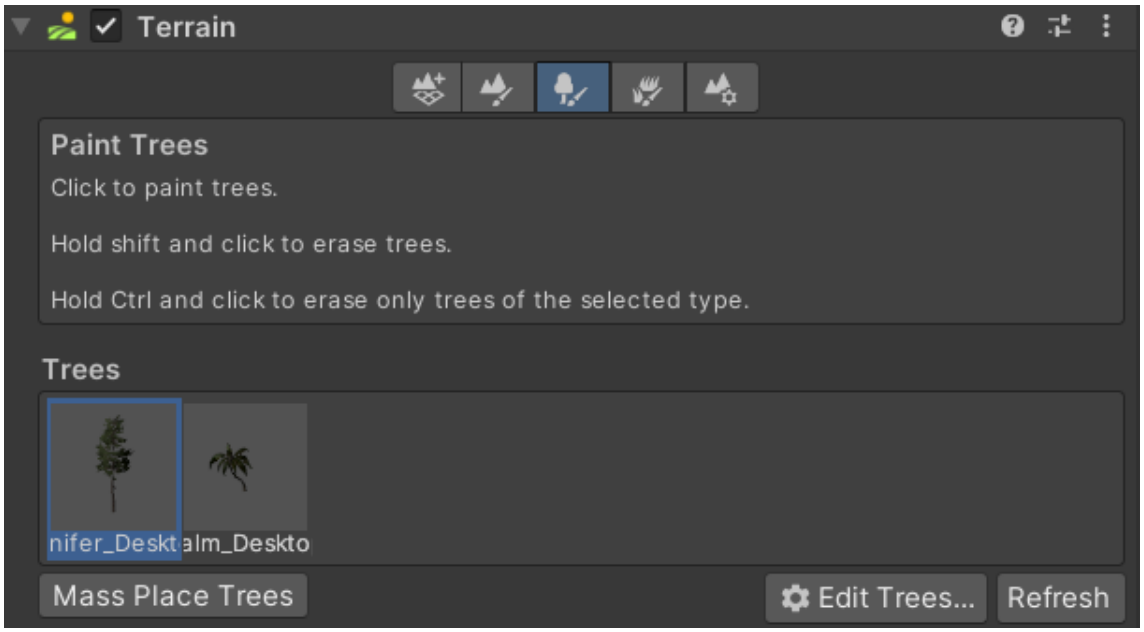




Con la misma herramienta de Paint Details, cambiando entre una textura y otra, he ido poniendo hierba sobre el terreno con la brocha, ajustando el tamaño y la opacidad para pintar la hierba de forma más o menos densa.



Hacemos lo mismo para los árboles, pero ahora con la herramienta Paint tres, y seguimos el mismo proceso que para las texturas de hirbas. Yo he usado los prefabs Conifer_Desktop y Palm_Desktop de la carpeta StandardAssets->Enviroment->SpeedTree.



Finalmente, el terreno me ha quedado algo así:

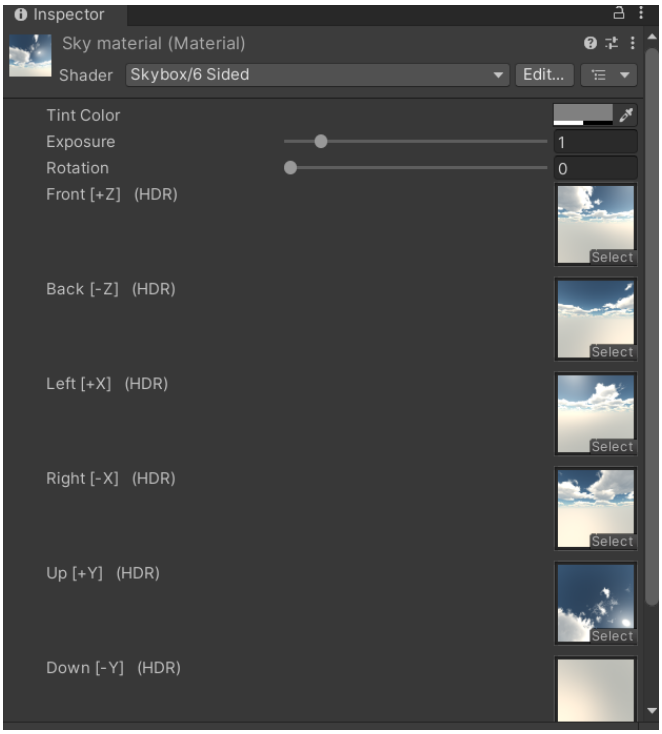


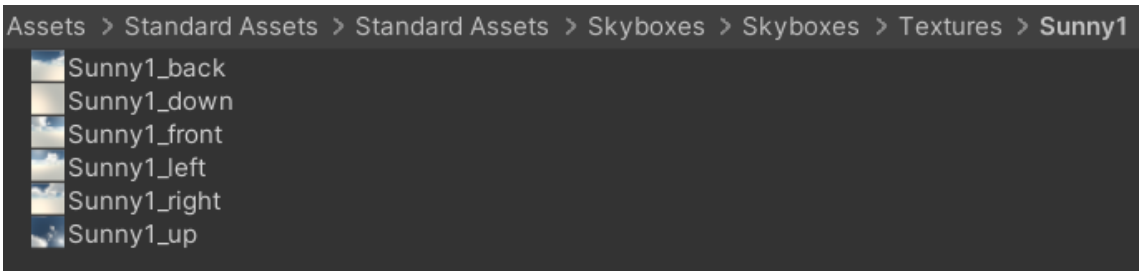
Nota: La hierba no se aprecia al estar tan alejado de ellas

Skybox

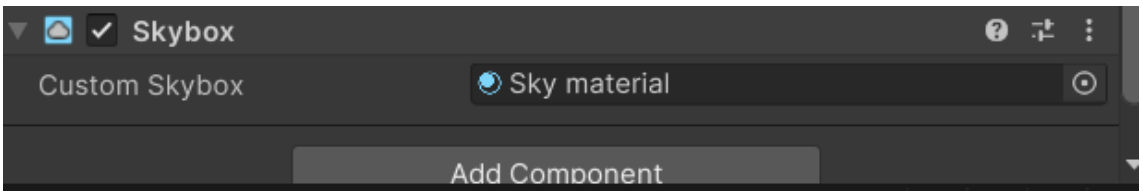
Para esta parte, he usado la carpeta de SkyBoxes del StandardAssets.

En primer lugar, en mi carpeta de materials he creado un nuevo material y lo he llamado Sky material, tras esto, en el Shader del material he seleccionado la opción de skybox->6 sided. Una vez hecho esto he aplicado cada textura correspondiente a cada cara. En mi caso he usado las texturas que están en la carpeta StandardAssets->Skyboxes->Textures->Sunny1.



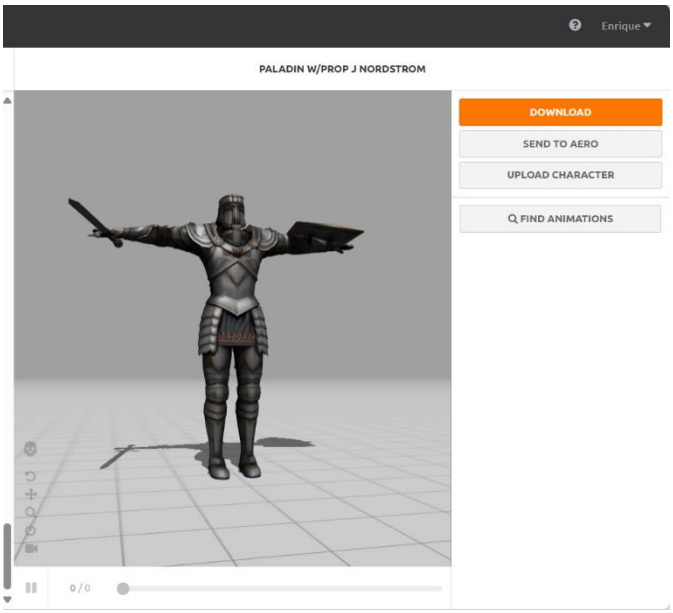


A continuación, en la cámara main, he añadido un componente de tipo Skybox y le he asignado el material de skybox que habíamos creado en el paso anterior.



MODELO Y ANIMACIONES DEL PERSONAJE PRINCIPAL

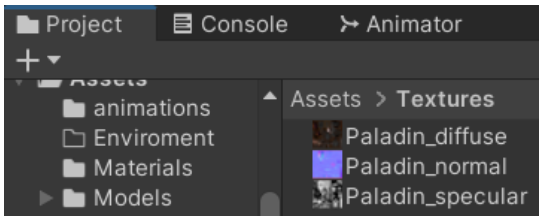
Para el personaje principal he elegido un modelo de mixamo llamado Paladin W/Prop J Nordstrom.

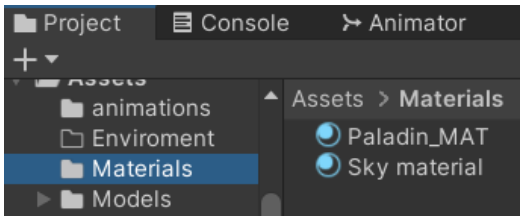


A continuación lo he descargado, marcando la opción de for Unity, y en T-Pose.

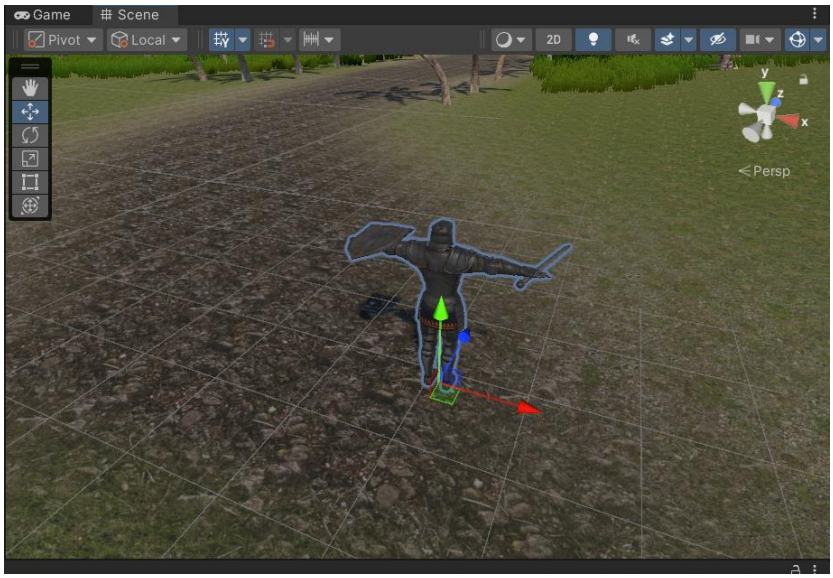
Una vez descargado, lo arrastro a la carpeta de Models en unity.

Como aparece sin texturas ni materiales, le damos a la opción de extract textures y extract materials,





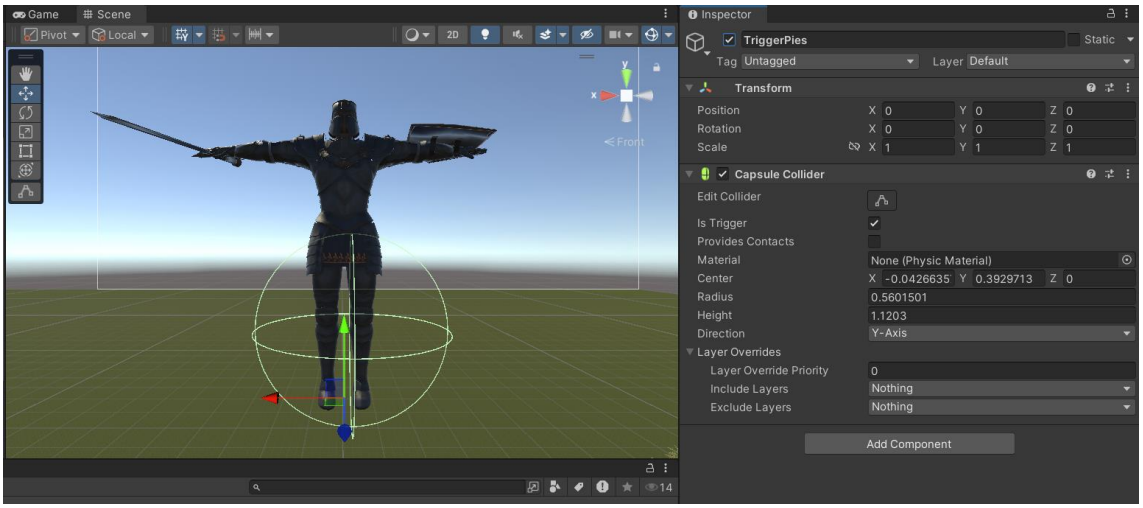
Ahora, si arrastramos el modelo al escenario ya aparece con sus texturas y materiales, en T-Pose.



Tras esto, le añado un RigidBody, congelándole la freeze rotation en los tres ejes, y le añado también un capsule collider y lo editamos para que se ajuste más o menos a su cuerpo. También le he añadido el script llamado LogicaPaladin, el cual contiene la mayoría de su funcionalidad que iremos viendo poco a poco.



También he creado un nuevo objeto, el cual se llama TriggerPies, y lo he arrastrado dentro del personaje para que lo siga y tenga la posición aproximada de sus pies. En este objeto he creado un sphere collider con la opción de trigger activada, para que podamos detectar si el personaje está tocando en suelo o no. Esto nos servirá más adelante para la funcionalidad de salto del personaje.



Como nuestro personaje va a atacar con una espada, y defenderse con un escudo, he creado otros dos GameObjects, uno llamado espada, al cual le he asigando un script

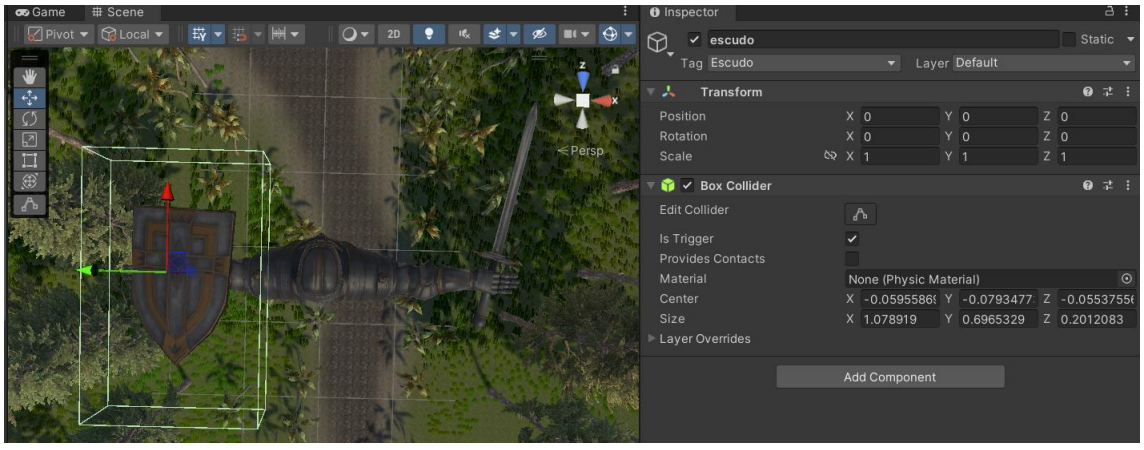
llamado danioPaladin, que controlará el tema del trigger de la espada cuando entra en contacto con el enemigo, y el otro llamado escudo, y como es necesario que se muevan junto con las animaciones, los he asignado a sus respectivas partes del cuerpo del personaje

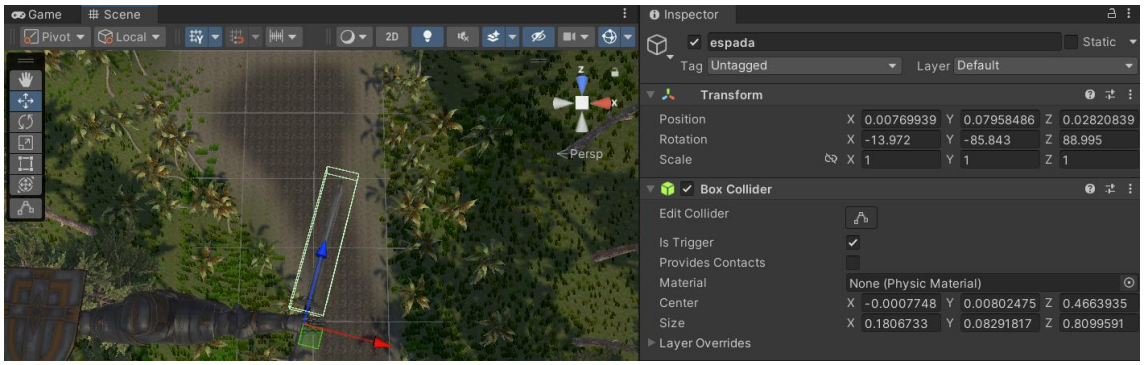


Como el escudo está en la mano izquierda, el GameObject escudo lo he arrastrado a la mano izquierda para que siga a esta cuando el personaje se mueva.

Lo mismo he hecho con el GameObject espada pero en la mano derecha.

Ahora, para que la espada y el escudo puedan interactuar con un enemigo, les añado un box collider y los ajusto mas o menos al tamaño de cada uno. Tras esto marcamos la opción de trigger en ambos colliders.





Tras esto dejo desactivado los collider del escudo y la espada, ya que la idea es que lo active y desactive mediante el script y eventos.

Animaciones del personaje principal

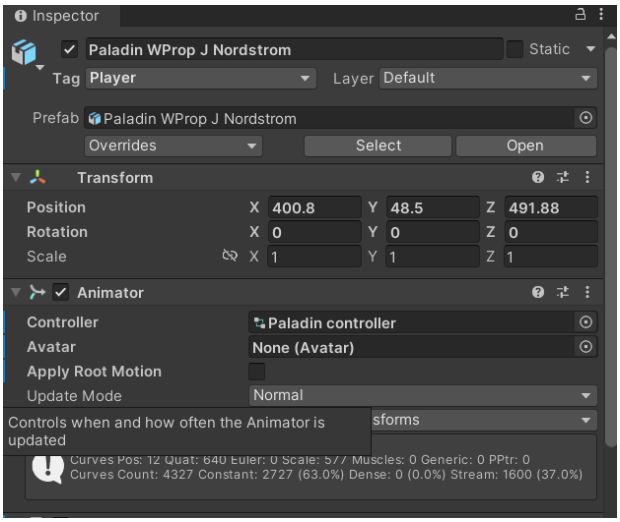
Para ponerle las animaciones al personaje, lo primero que he hecho es crear una carpeta “animations” y guardar en ella todas las animaciones que he descargado de mixamo:



Y para poder implementar estas animaciones, he creado un animator controller llamado Paladin Controller, lo he guardado en la carpeta de animations y se lo he asociado al personaje.

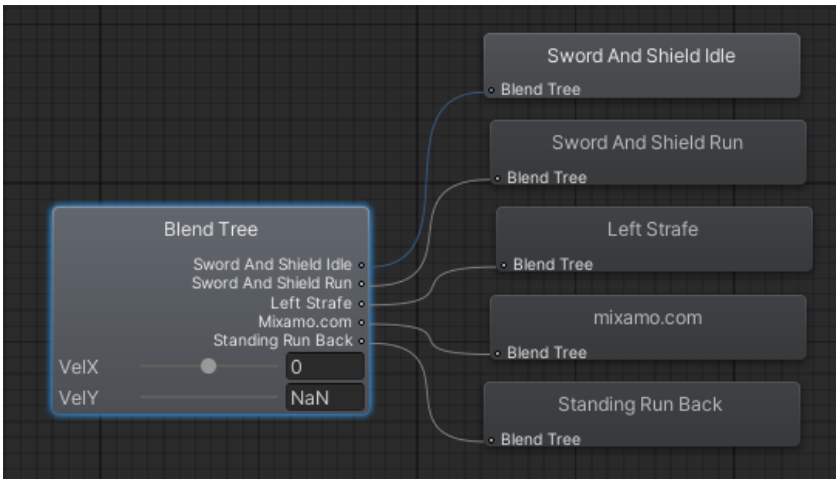
Animator “Paladin Controller”

Una vez con todas estas animaciones descargadas, para poder implementarlas, he creado un animator controller llamado Paladin Controller, lo he guardado en la carpeta de animations y se lo he asociado al personaje.

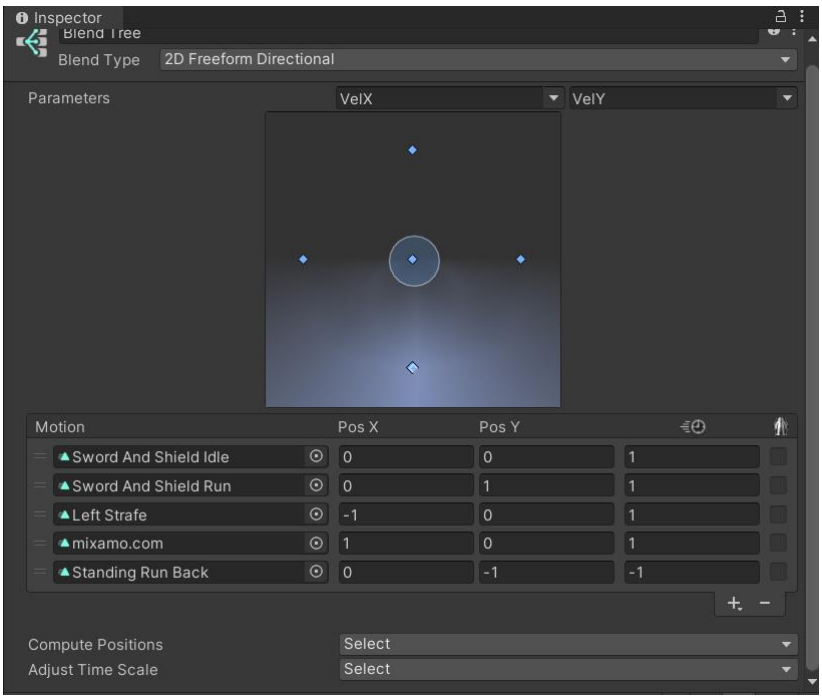


Blend tree

Dentro del animator, creo dos parámetros VelX y VelY, que me servirán para implementar la lógica de movimiento del personaje. También he creado un blend tree para la movilidad del personaje, en el cual voy a meter las animaciones de correr hacia adelante, atrás, izquierda, derecha y la de parado. Este blend tree lo he conectado con el entry.



A continuación, en el inspector introducimos para qué valores de VelX y VelY se tiene que activar cada animación.

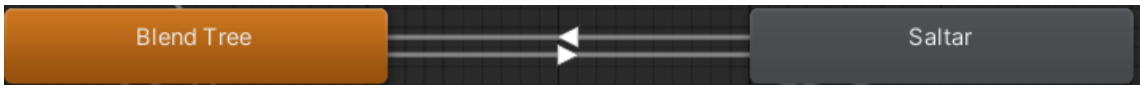


Animación de salto

Para implementar la animación de salto, he creado dos parámetros en el animator, onGround, que es un bool que nos indica si estamos tocando el suelo o no, y Saltando, que es otro bool que nos indica si estamos saltando o no.

También he creado un nuevo estado llamado saltar, a el cual le he asignado la animación de salto.

Tras esto, creo las transiciones que van a ir del blend tree al estado saltar y viceversa.



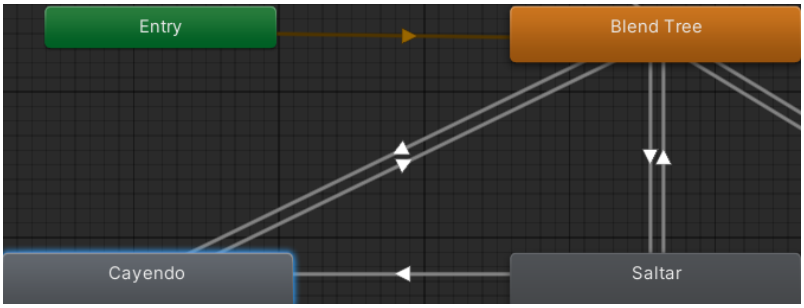
En la transición blend tree->Saltar, la condición es si Saltando = true, y le he desactivado la opción de Has exit time. Para la transición de Saltar->Blend tree la condición es si OnGround = true, también sin el has exit time.

Para detectar si estamos en el suelo o no, entra en juego el GameObject triggerPies que hemos mencionado antes. En el script LogicaPaladin, encontramos dos métodos, OnTriggerStay y OnTriggerExit, los cuales se refieren a este triggerPies. OnTriggerStay pone OnGround a true y Saltando a false mientras el trigger está activo.

OnTriggerExit pone a onGround en false cuando triggerPies no detecta otro collider.

Animación de caída

Para la animación de caída, he creado un nuevo estado llamado Cayendo, en el cual está asignada la animación de caída. Está unido con los estados Saltar, y blend tree.

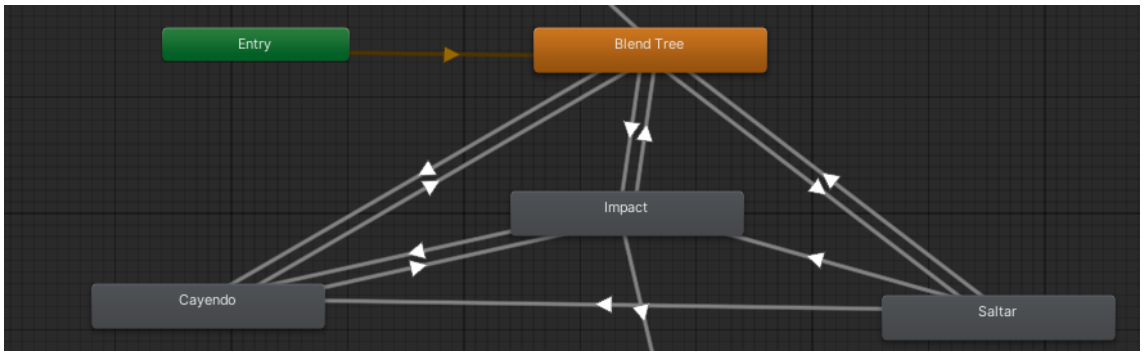


Para la transición de Saltar->Cayendo he puesto la condición onGround = false, para la transición Cayendo->Blend tree la condición onGround = true, y para Blend Tree->Cayendo la condición onGround = false. Todas estas sin has exit time.

Esta animación también depende del trigger triggerPies para que active o desactive las transiciones dependiendo de si triggerPies está en contacto con algo o no.

Animación de impacto en el jugador

Es la animación que representa cuando el enemigo golpea a nuestro jugador. Para ello he creado un parámetro Impact de tipo bool y un nuevo estado Impact. En dicho estado he colocado la animación de impacto y he unido dicho estado con todos los demás, ya que quiero que salte la animación estando en cualquiera de los estados.

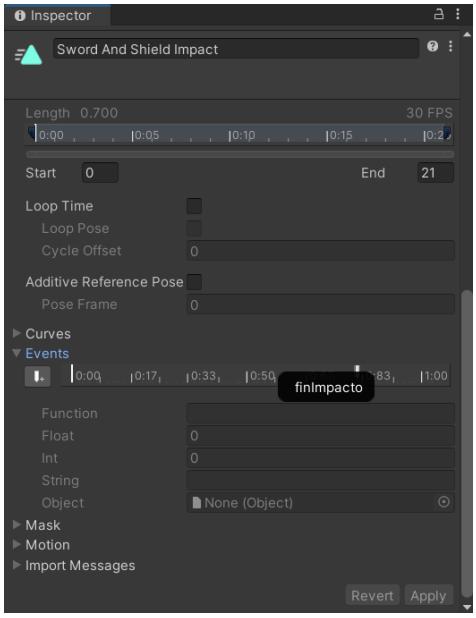


Todas las transiciones que entran en el estado Impact se producen cuando el parámetro Impact se pone en true. La transición Impact->Blend tree ocurre cuando impact = false y onGround = true. La transición de Impact->Cayendo ocurre cuando Impact = flase y OnGround = false;

En el script danioEnemigo, que está asociado al gameObject jointColliderAttack, tenemos la función OnTriggerEnter, la cual, cuando el trigger que hace de puño en el enemigo, detecta un collider, compara el tag del collider con el que ha interactuado y

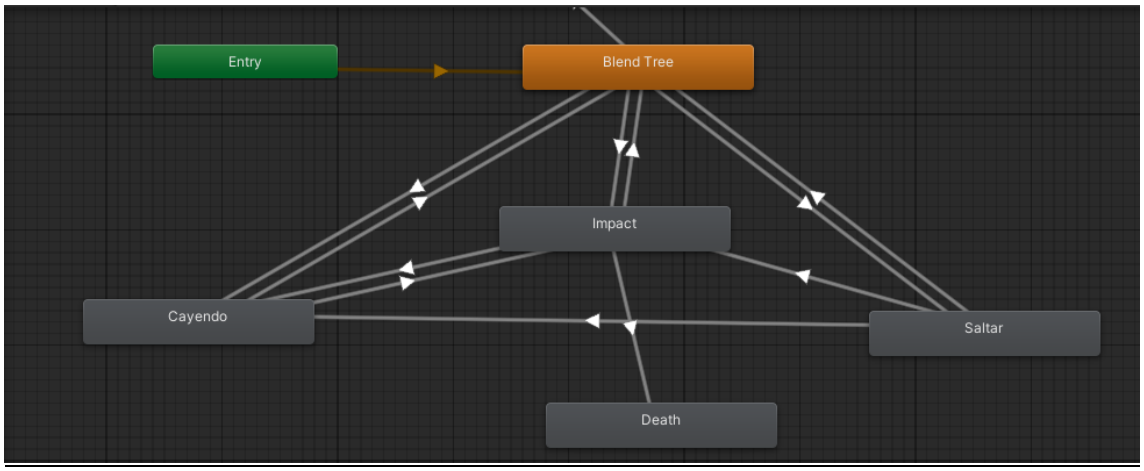
si interactúa con el collider del jugador, cambia el valor del parámetro Impact del animator del jugador a true, pasando el animator del jugador al estado de Impact.

En la animación de impacto en el personaje, en las propiedades->Events, encontramos un evento casi al final de la animación llamado finImpacto, que es una función que se contiene en el script LogicaPaladin, la cual vuelve a poner el parámetro de Impact a false.



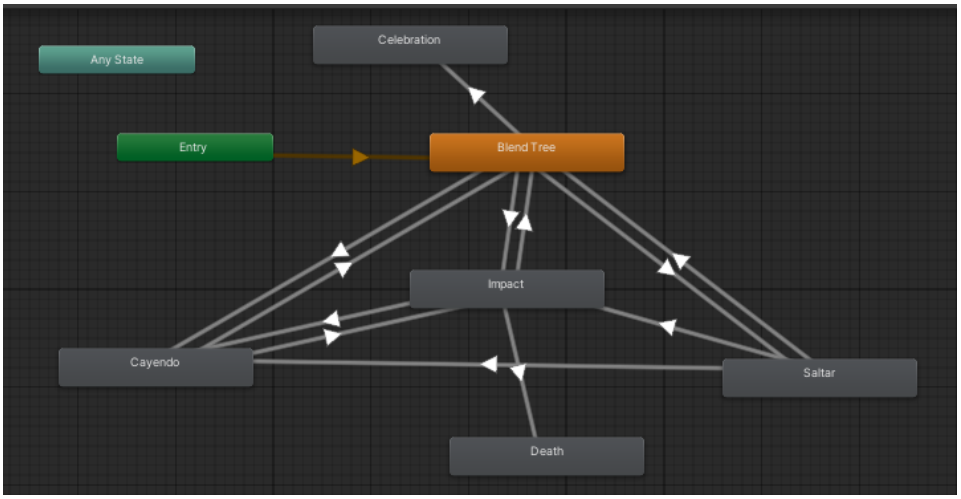
Animación de muerte en el personaje

Esta animación se va a reproducir cuando la vida del personaje sea menor o igual a 0. Creo un nuevo parámetro Death de tipo bool, y un nuevo estado llamado Death. Este estado solo tiene transición de entrada, es la transición Impact->Death, ya que antes de que el personaje llegue a 0 de vida tiene que pasar por el estado de Impact debido al golpe que le ha quitado la vida que le quedaba. Esta transición de Impact->Death se produce cuando el parámetro Death = true;



Animación de celebración

Esta animación representa la victoria del personaje, y sólo se reproducirá una vez que el personaje haya eliminado a todos los enemigos, parecido al caso de Death. He creado un nuevo parámetro Celebration y un nuevo estado Celebration.

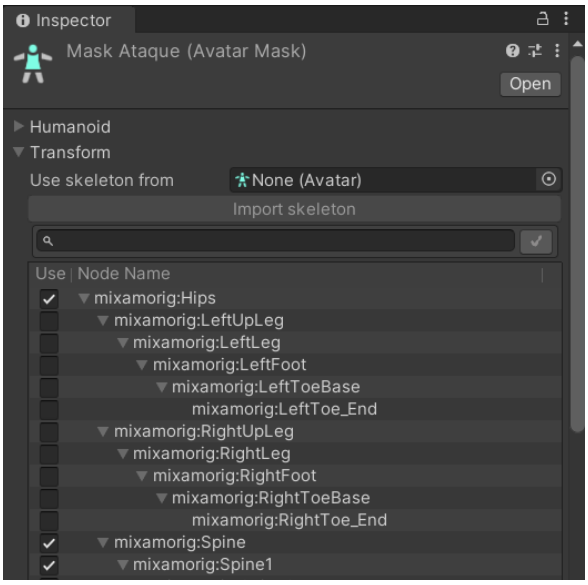


He creado la transición de Blend Tree->Celebration ya que solo quiero que celebre el personaje si no está ni saltando, ni cayendo.

Esta transición se reproducirá si el parámetro Celebration = true.

Animación de ataque

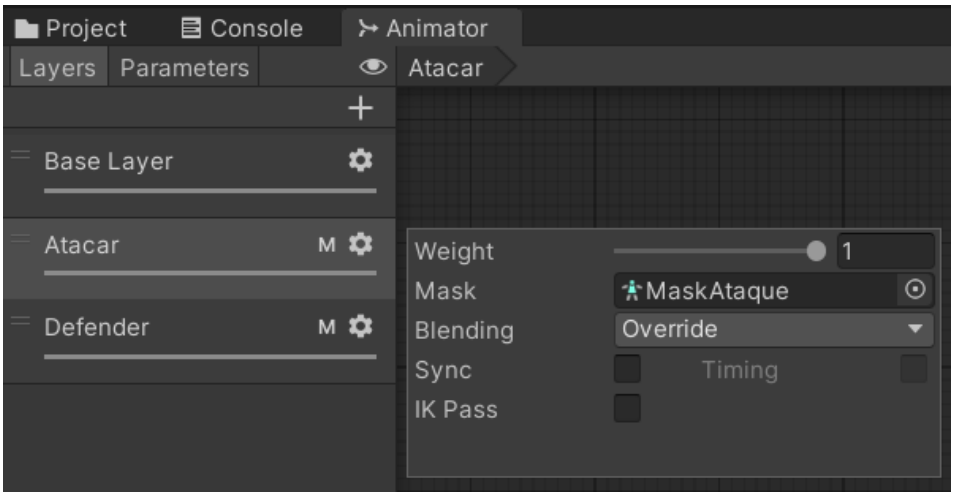
Esta animación es la de ataque del personaje. Pero había un problema, la animación es un ataque con las piernas paradas, y lo más acertado sería que si ataco mientras me estoy moviendo, el personaje no se mueva con las piernas paradas a la vez que ataca. Con lo cual la única solución que he encontrado es crear una avatarMask llamada maskAtaque, mediante la cual activar y desactivar los huesos que van a recibir la animación, y una nueva capa en el animator en la que se aplique dicha máscara. Con esto he conseguido que la animación de atacar solo se aplique a la parte de arriba del cuerpo, y la parte de abajo del cuerpo siga la animación que le corresponda en la otra capa.



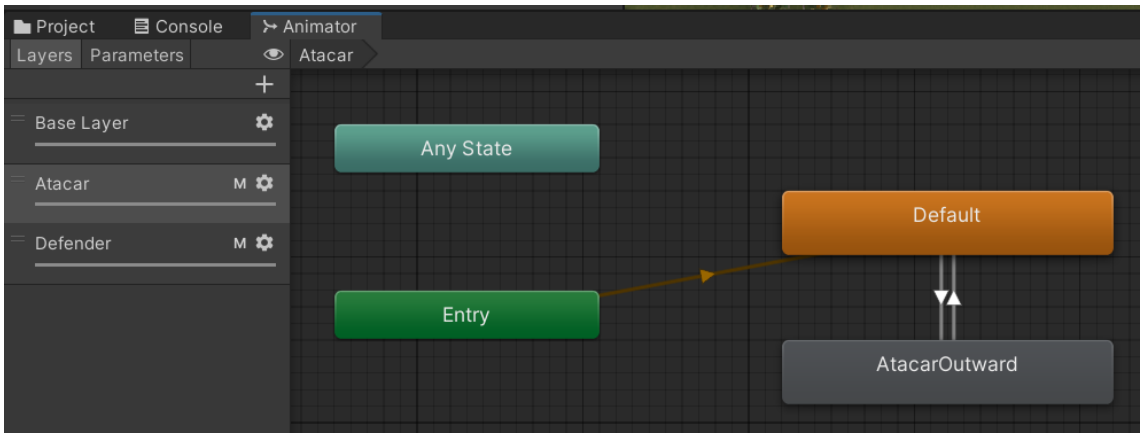
En primer lugar, en la carpeta de animations, he creado el avatarMask, y en la opción de Transform, he desactivado los huesos que correspondían a la parte de abajo del cuerpo, para que no reciban la animación de ataque.

En esta imagen se puede ver los huesos que he desactivado.

En segundo lugar he creado un nuevo layer llamado Atacar en el animator, le he asigando la máscara, he marcado la opción de override en el blending y le he puesto el weight al máximo.

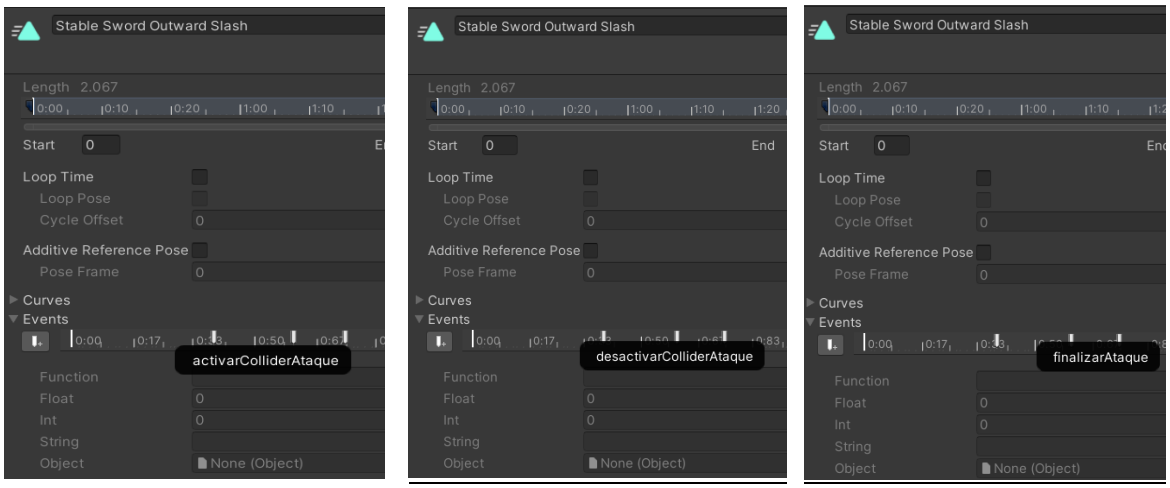


Ahora toca crear los estados que van a pertenecer a esta capa del animator. He creado el parámetro bool Atacar, dos estados, el primero, que va conectado al Entry sin ninguna condición, es un estado vacío la que he llamado Default. El animator va a estar en este estado todo el rato hasta que decidamos atacar. El segundo estado es el estado de AtacarOutward, al que le he asignado la animación de ataque. He creado 2 transiciones entre Default y AtacarOutward, la transición Default->AtacarOutward se activa si el parámetro Atacar = true, y la transición AtacarOutward->Default se activa si AtacarOutward = false.



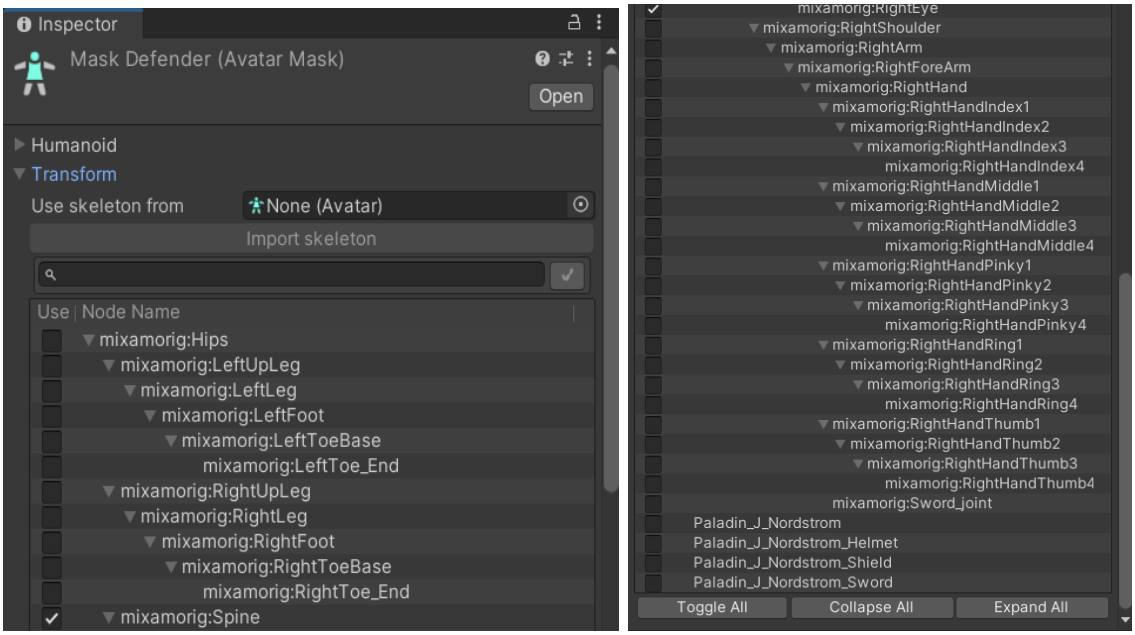
La animación de ataque contiene 3 eventos, el primero activa el collider de la espada llamando a la función activarColliderAtaque en un determinado tiempo de la animación de ataque, esto es necesario para que el collider no detecte al enemigo si entran en contacto los colliders sin estar atacando. El segundo desactiva el collider de la espada llamando a la función desactivarColliderAtaque para lo que acabamos de mencionar. Y el tercer evento vuelve a poner el parámetro de Atacar a false cuando está acabando la animación, llamando a la función finalizarAtaque.

Estas tres funciones a la que llaman los eventos se contienen en el script LogicaPaladin, ya que el script en el que están debe estar en el mismo gameObject que el animator desde el que se lanza el evento, sino daba error.



Animación de defensa e impacto en el escudo

En estas animaciones tenía el mismo problema que en la de ataque, con lo cual he creado otra máscara llamada maskDefender y otra capa en el animator siguiendo los mismos pasos que en la de ataque.

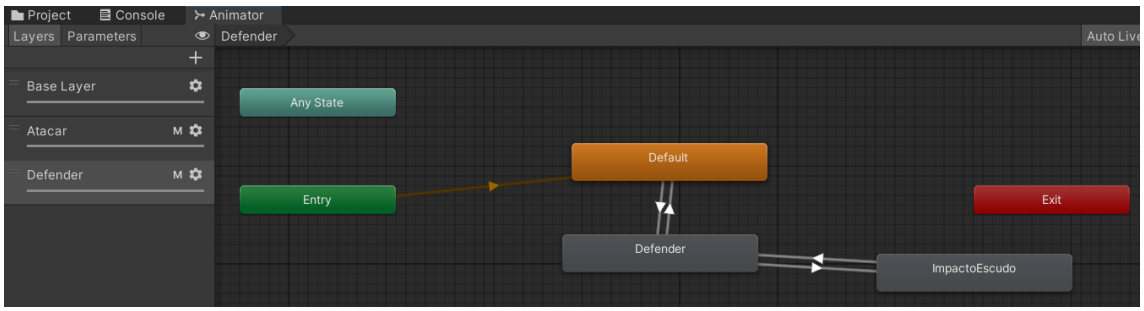


En estas imágenes muestro los huesos del personaje que no van a recibir la animación de defensa con el escudo. Ahora he desactivado los del brazo derecho también, ya que quería que la espada se moviese como cuando se reproduce las animaciones del blend tree.

Ahora, en esta capa del animator he creado 3 estados, el estado Default, que sigue la misma funcionalidad que el de la capa Atacar, el estado Defender el cual tiene asignada

la animación de defender con el escudo, y un tercero llamado ImpactoEscudo, que nos va a servir para representar cuando el enemigo impacta en el escudo.

Junto con estos estados, he creado 2 parámetros de tipo bool más, Defender e ImpactoEscudo.



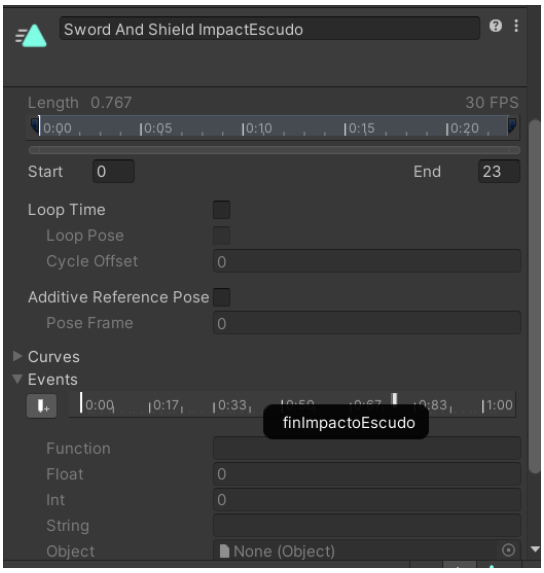
Transiciones:

- Entry->Defaul: no tiene condición asique entra directamente
- Defaul->Defender: si defender = true
- Defender->Default: si defender = false
- Defender->ImpactoEscudo: si ImpactoEscudo = true
- ImpacoEscudo->Defender: si ImpactoEscudo = false

En el script LogicaPaladin, se activa el collider del escudo cuando se mantiene el click derecho del ratón, y se desactiva cuando se suelta.

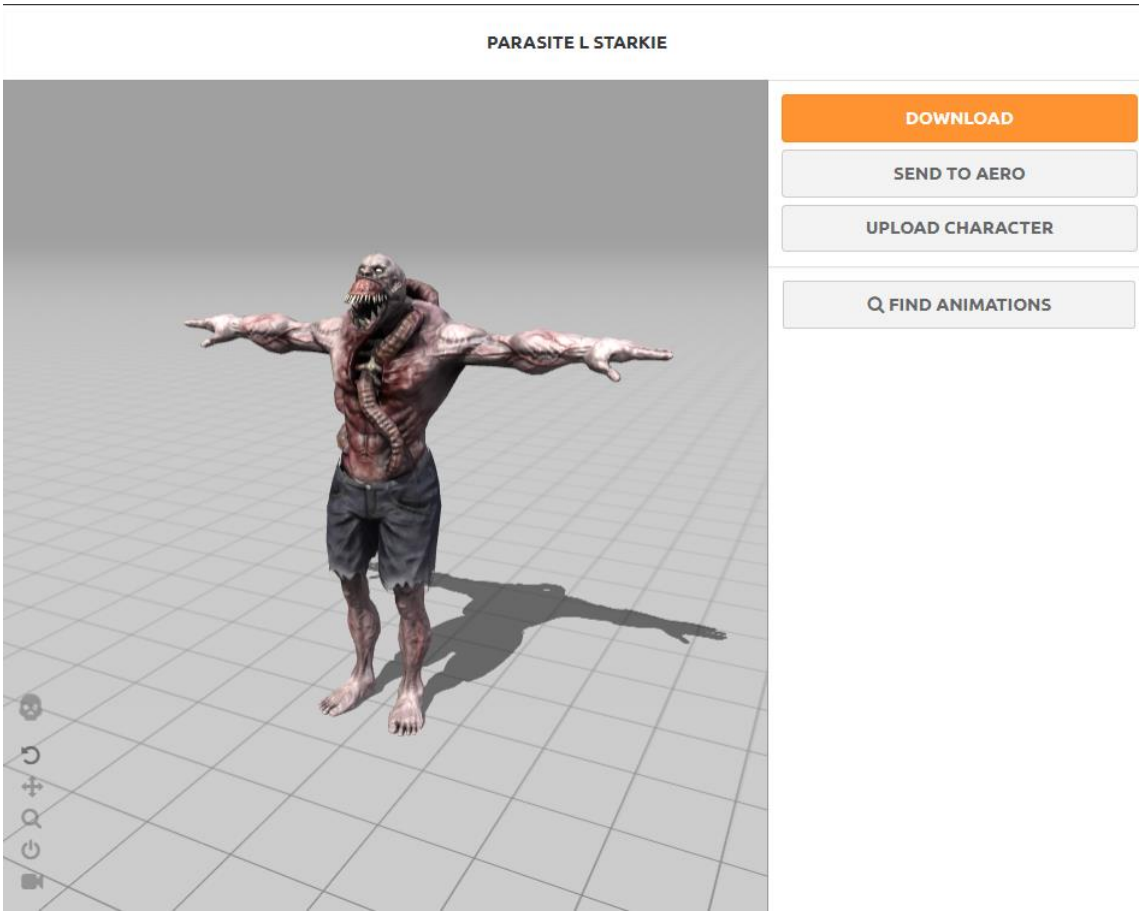
En el script danioEnemigo, que está asociado al gameObject jointColliderAttack, tenemos la función OnTriggerEnter, la cual, cuando el trigger que hace de puño en el enemigo, compara el tag del collider con el que ha interactuado y si interactúa antes con el collider del escudo que con el del jugador, cambia el valor del parámetro ImpactEscudo del animator del jugador a true, pasando el animator del jugador al estado de ImpactEscudo. Además de esto, al entrar en contacto con el collider del escudo, el collider del puño del enemigo se desactiva para que no colisione con el jugador después de tocar el escudo.

En la animación de impacto escudo, tenemos un evento el cual llama a la función finImpactoEscudo casi al final de la animación. Dicha función pone el parámetro impactoEscudo a false cuando salta el evento y está contenida en el script LogicaPaladin.



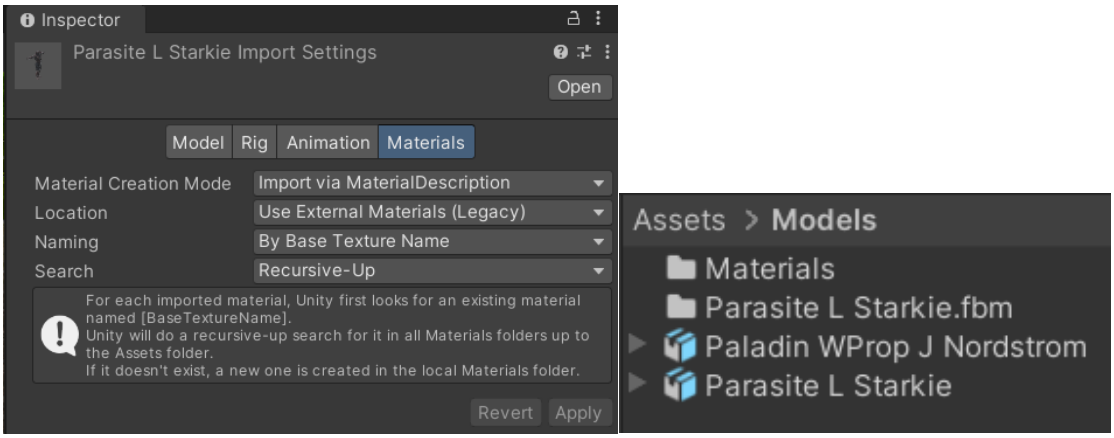
MODELO Y ANIMACIONES DEL ENEMIGO

Para los enemigos, he buscado un modelo en Mixamo y finalmente he elegido uno llamado Paasite L Starkie. Es como una especie de monstruo o zombie que me servirá como enemigo.

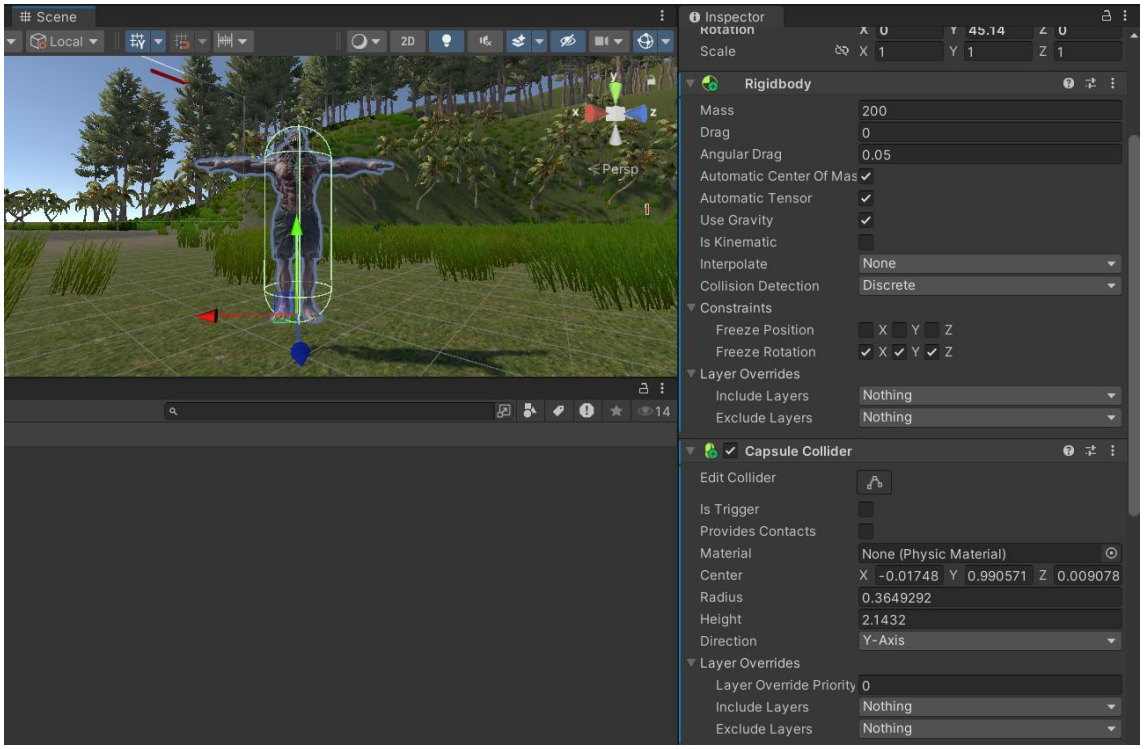


Lo descargo para Unity y en T-pose. Para usar este modelo he seguido el mismo procedimiento que para el personaje del jugador, pero esta vez en vez de extraer las

texturas y los materiales y guardarlos en sus respectivas carpetas, he pulsado la opción de use external materials. Con esto se me han extraído las texturas y los materiales de forma automática y se contienen en la carpeta models.

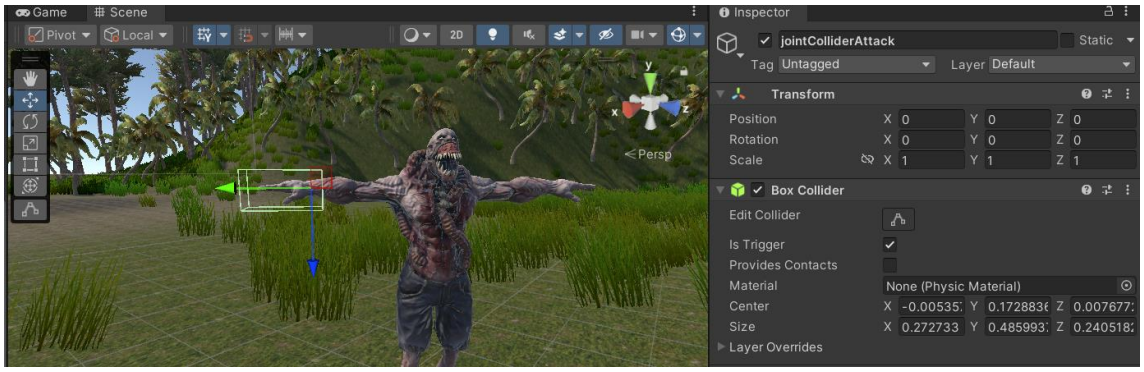


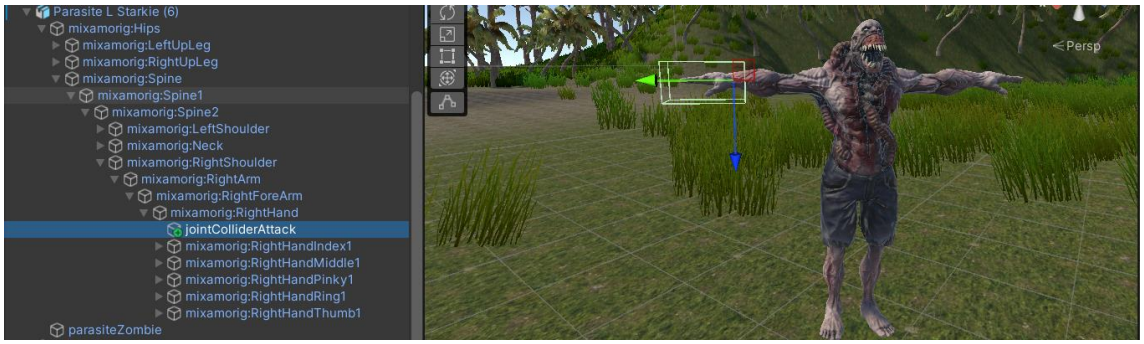
Ahora arrastro el modelo del enemigo a la escena y le añadimos el rigidBody y el capsule collider que le corresponde. También le añado el script de LogicaEnemigo, el cual se va a encargar de la lógica de movimiento y la transición entre estados.



Como en el modelo del jugador, necesitamos ponerle un collider con la opción de trigger activada en la parte del cuerpo con la que vaya a atacar. Para ello he creado un nuevo GameObject llamado jointColliderAttack, el cual lo he arrastrado a la mano derecha del enemigo y le he ajustado el collider al tamaño aproximado de la zona con la que golpea. Este GameObject va a tener asociado el script danioEnemigo, el cual se va a encargar de la lógica del trigger del puño, comprobando el collider con el que

impacta. Dependiendo de cual sea el collider con el que collisiona, Player o Escudo, hará una cosa u otra.

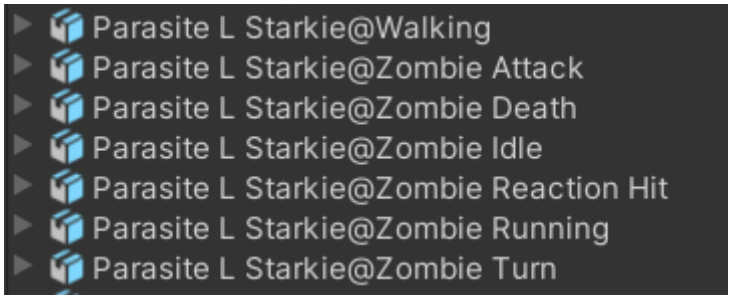




Tras esto dejo desactivado el colllider del puño, ya que la idea es que lo active y desactive mediante el script y eventos.

Animaciones del enemigo

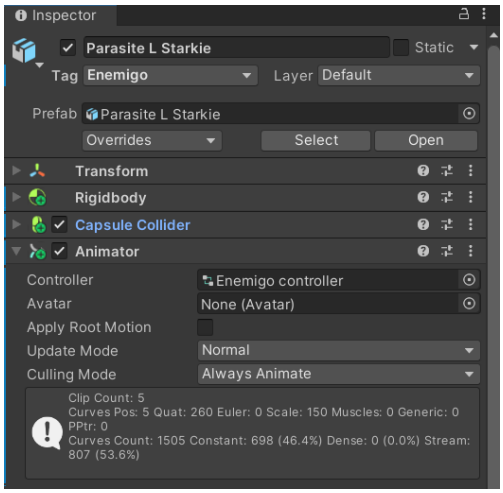
Una vez configurado el modelo del enemigo, me he descargado varias animaciones para el enemigo. La mayoría son de movilidad básica para que no tenga tantas capacidades como el jugador. Estas son las animaciones que me he descargado para el enemigo:



Todas estas animaciones están en la carpeta animations, junto con las del jugador.

Animator "Enemigo controller"

Para implementar estas animaciones en el enemigo he creado un nuevo Animator controller dentro de la carpeta animations, y se lo he asignado al modelo del enemigo.



Animación de andar del enemigo

He pensado que, como primer estado y por defecto del enemigo, sea el estado deambular. Para implementar este estado he pensado en que el enemigo se mueva siempre hacia adelante con una velocidad constante y cada cierto tiempo haga un giro de 90° a la derecha para que se mantenga siempre sobre la misma zona, haciendo como una especie de cuadrado como recorrido de deambulación.

Para ello, he creado el estado deambular en el animator y lo he conectado con el entry.



Dentro del estado deambular le asigno la animación de andar, y creo un parámetro de tipo bool llamado Deambulando el cual va a ser el único que esté en true antes de iniciar el juego, para que cuando empecemos, los zombies se pongan a deambular automáticamente.

Animación de perseguir al jugador

Como segundo estado, he implementado la funcionalidad de perseguir al enemigo si este se acerca lo suficiente al zombie. Para esto he creado el estado Perseguir con su respectivo parámetro de tipo bool. En el estado Perseguir asignamos la animación y creamos las transiciones entre deambular y perseguir.



Transiciones:

- Deambular->Perseguir: con la condición de Perseguir=true
- Perseguir->Deambular: con la condición de Perseguir=false

Sólo transitará de perseguir a deambular cuando el zombie mate al jugador, es decir, no parará de perseguirlo hasta que lo mate o muera el zombie.

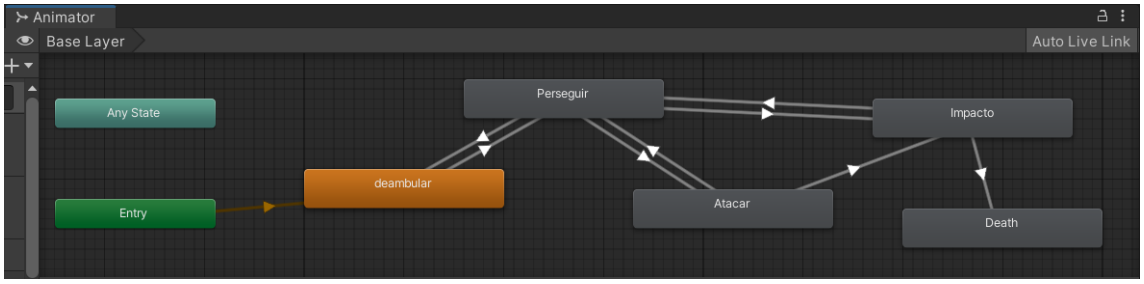
Para conseguir que el enemigo persiga al personaje he seguido varios pasos. Una vez que el jugador invade la distancia establecida para que el zombie empiece a perseguir al jugador, se aumenta la velocidad de movimiento del enemigo para que se note la diferencia de deambular a perseguir. También creo una variable dirección de tipo Vector3 y consigo la dirección restando la posición del enemigo a la del jugador. Creo también una variable rotation de tipo Quaternion que va a representar la rotación que hay que aplicar al enemigo para que mire hacia el jugado. Una vez consigo la dirección con la resta de posiciones, obtengo el valor de la variable rotation con un Quaternion.LookRotation(direccion). Una vez conseguida la rotación que hay que aplicar, la aplicamos al transform del enemigo haciendo:

transform.rotation = Quaternion.RotateTowards(transform.rotation, rotation, VelRotacion * Time.deltaTime).

Todo esto a la vez que el enemigo avanza constantemente hacia adelante.

Animación de impacto en el enemigo

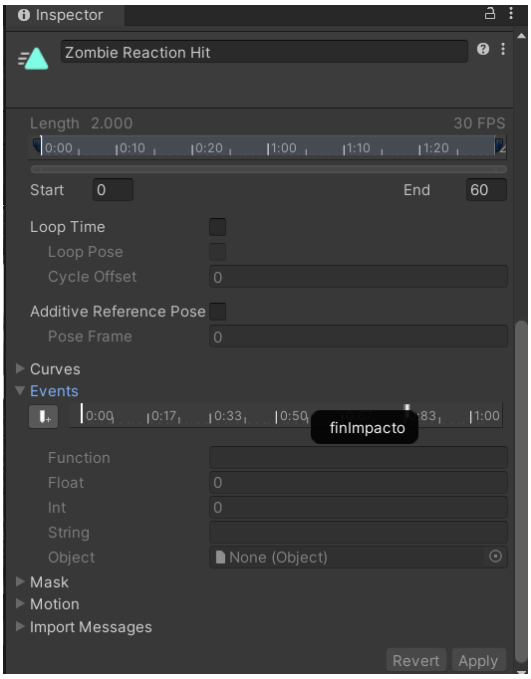
Creo un nuevo estado llamado Impacto y un parámetro de tipo bool llamado Impact. Asigno la animación de impacto al estado y creo las transiciones con el estado de perseguir.



Transiciones:

- Perseguir->Impacto: ocurre cuando Impact = true. Este parámetro se pone a true en la función OnTriggerEnter del script danioPaladin, el cual está asociado al GameObject espada en el personaje del jugador. Si el trigger de la espada collisiona con otro collider, comprueba el tag y si es el tag del enemigo entonces pone el parámetro de Impact a true en el animator del enemigo.
- Impacto->Perseguir: ocurre cuando impacto = false. Este parámetro se pone a false cuando el evento de la animación de impacto llama a la función finImpacto

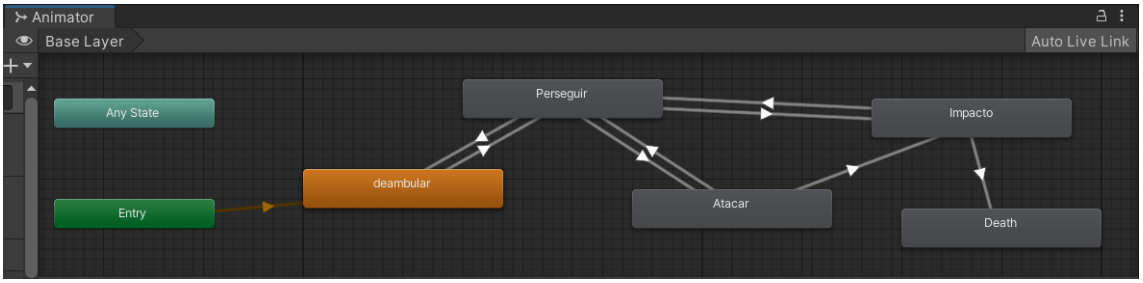
la cual está en el script de LogicaEnemigo. Cuando la animación llama con el evento a finImpacto, esta función pone el parámetro impacto en false, y el de Perseguir en true.



- Atacar->Impacto: ocurre cuando impacto = true. Lo puse así porque mientras ataca el enemigo puede recibir un golpe y debe reaccionar, tras esto se cancela el golpe y tiene que volver a atacar.
- Impacto->Death: ocurre cuando el parámetro Death = true, esto pasa cuando la vida del enemigo <=0.
-

Animación de muerte del enemigo

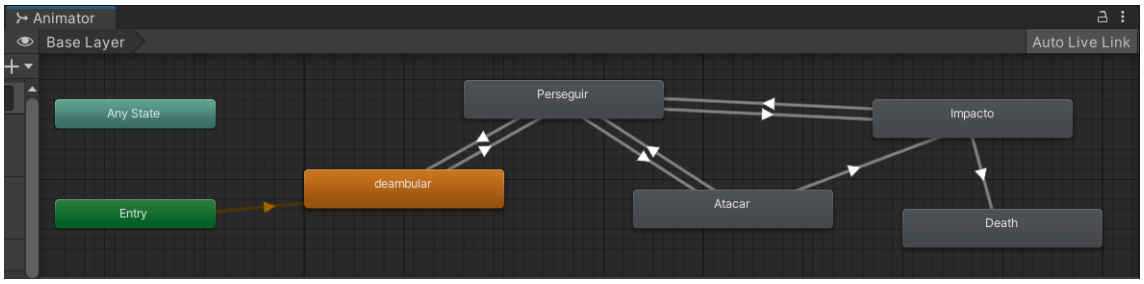
Creo el estado Death y su parámetro de tipo bool.



La transición de Impacto->Death ocurre cuando el booleano Death se pone a true, esto ocurre cuando la vida del enemigo <=0.

Animación de ataque del enemigo

Creo el estado Atacar y su parámetro de tipo bool.

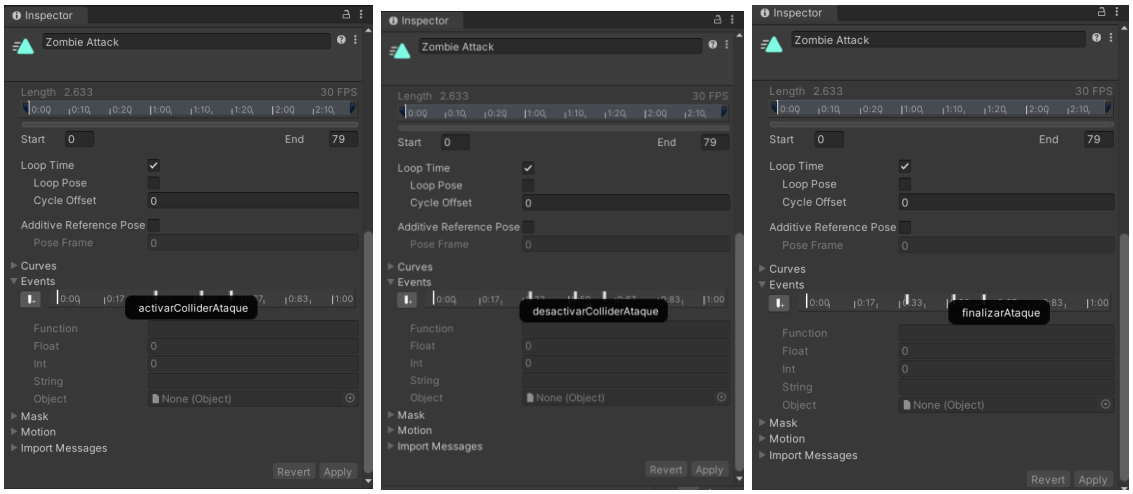


Transiciones:

- Perseguir->Atacar: cuando el bool atacar se pone en true. Esto ocurre cuando el enemigo está a una determinada distancia del jugador. Cuando llega a esa distancia se para y ataca.
- Atacar->Perseguir: cuando el bool atacar se pone en false y el enemigo ha salido de la distancia de ataque. Esto ocurre cuando en la animación de ataque se produce el evento casi al final de la animación que llama a la función finalizarAtaque que está contenida en el script de logicaEnemigo. En esta función de logicaEnemigo se comprueba la distancia a la que se encuentra el jugador del enemigo y si es mayor a la distancia de ataque pues pone el parámetro de perseguir a true, pero si la distancia sigue siendo menor o igual a la de ataque, no se cambia el parámetro para que el enemigo siga atacando.

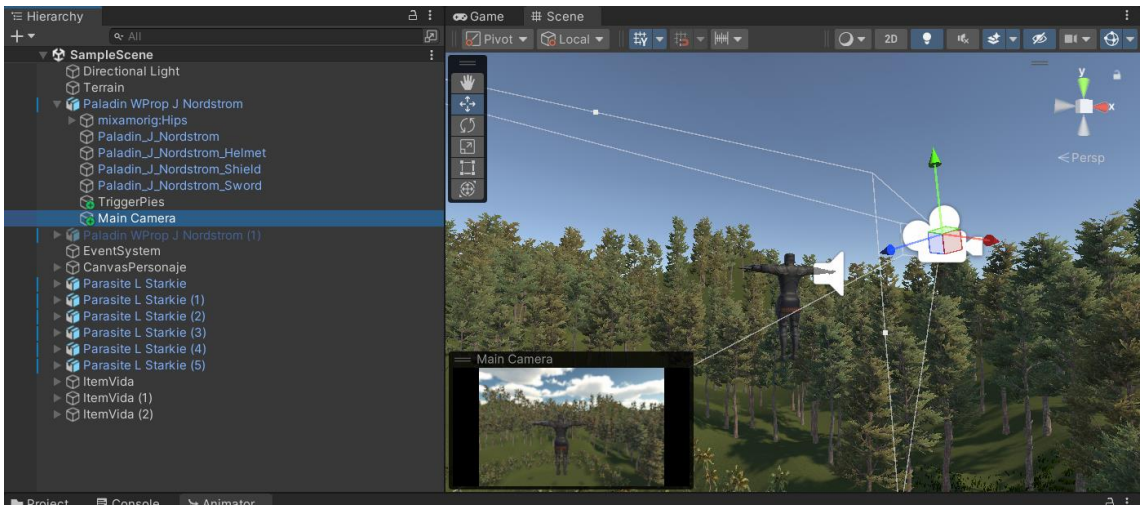
Eventos del estado atacar:

- 1° evento: el primer evento que se produce en la animación de ataque es el que llama a la función de activarColliderAtaque, la cual está contenida en el script de LogicaEnemigo. Esta función activa el collider del puño del enemigo en un determinado momento de la animación por el mismo motivo que mencionamos antes con la espada del jugador.
- 2° evento: llama a la función desactivarCollider del script LogicaEnemigo, el cual desactiva el collider del puño del enemigo por lo mismo que la espada, para que no collisione con el jugador si no le estamos atacando.
- 3° evento: llama a la función finalizarAtaque. En esta función de LogicaEnemigo se comprueba la distancia a la que se encuentra el jugador del enemigo y si es mayor a la distancia de ataque pues pone el parámetro de perseguir a true, pero si la distancia sigue siendo menor o igual a la de ataque, no se cambia el parámetro para que el enemigo siga atacando.



CÁMARA DEL JUEGO

Para la lógica de la cámara, lo primero que he hecho es arrastrar la cámara main al personaje del jugador para que le siga, luego la he orientado más o menos para que se vea lo que quiero.



La lógica está implementada en el script LogicaPaladin y consiste en que tengamos una variable rotMouseX la cual almacena el valor cuando movemos el ratón en el eje X mediante Input.GetAxis("MouseX").

Con este valor, aplicamos la rotación al personaje con

rotX = rotMouseX * Time.deltaTime * VelRotacion;

transform.Rotate(0, rotX, 0);

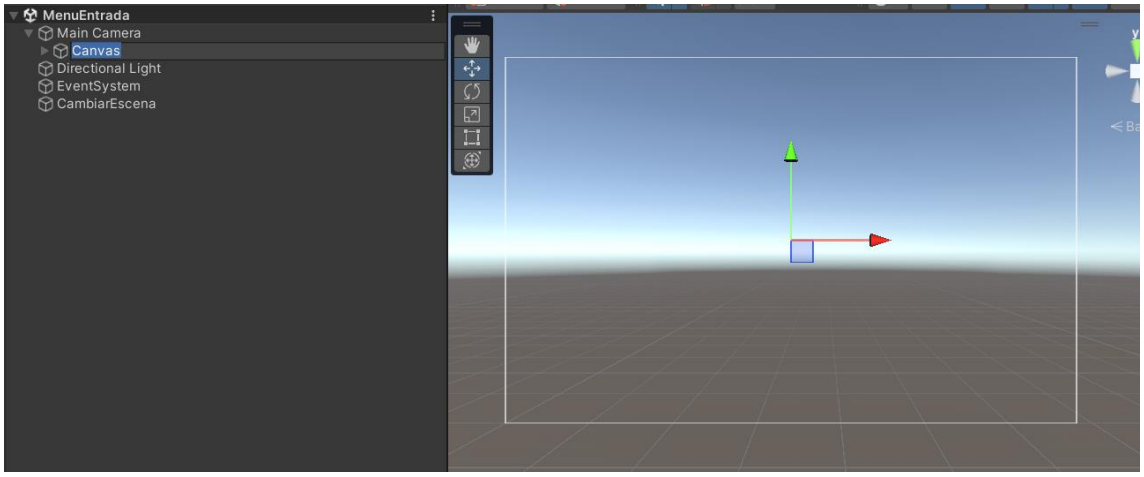
de forma que al mover el ratón en el eje X, el personaje rote hacia ese lado, y como la cámara está asignada al personaje del jugador, la cámara rota con él.

Sólo he hecho que se pueda rotar la cámara y jugador en el eje X, ya que siempre vamos a atacar a los enemigos en horizontal.

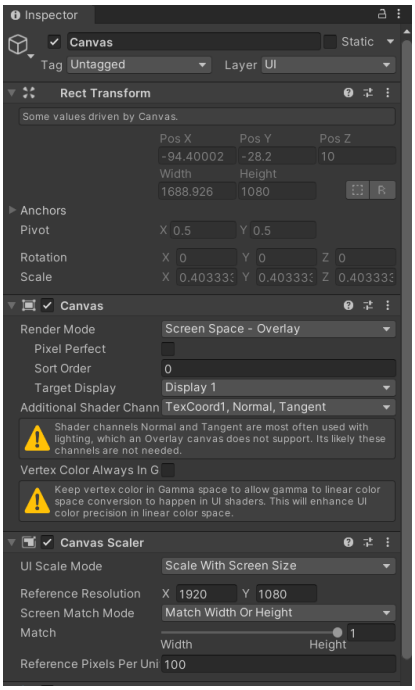
ESCENA "MENÚ DE ENTRADA"

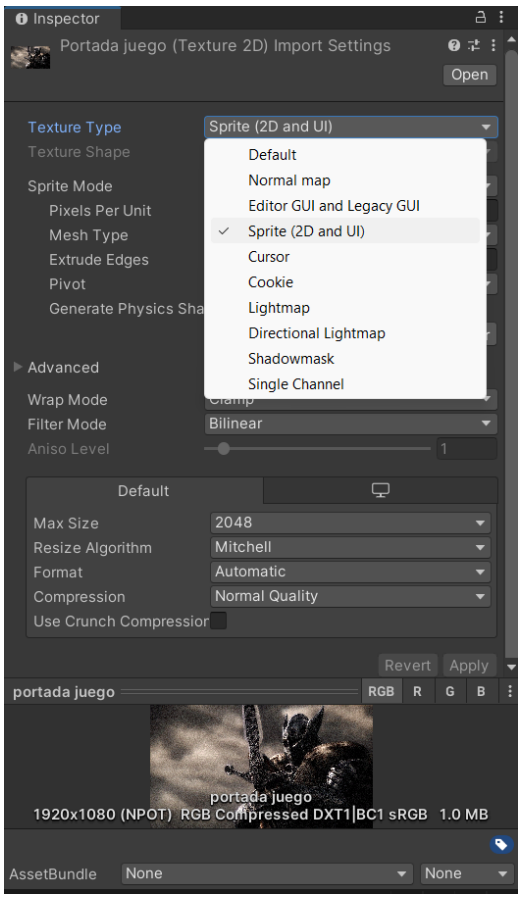
Esta escena es la que se va a ejecutar al entrar al juego. La idea es poner un canvas con una imagen de fondo y un botón para comenzar y cambiar a la escena de acción.

Para esto he creado una nueva escena y la he guardado en la carpeta scenes. Dentro de la nueva escena a la que he llamado MenuEntrada he creado un canvas y lo he arratrado dentro de la Main Camera.



He cambiado la configuración del canvas para que se ajuste al tamaño de la pantalla. A continuación muestro la configuración final del canvas:

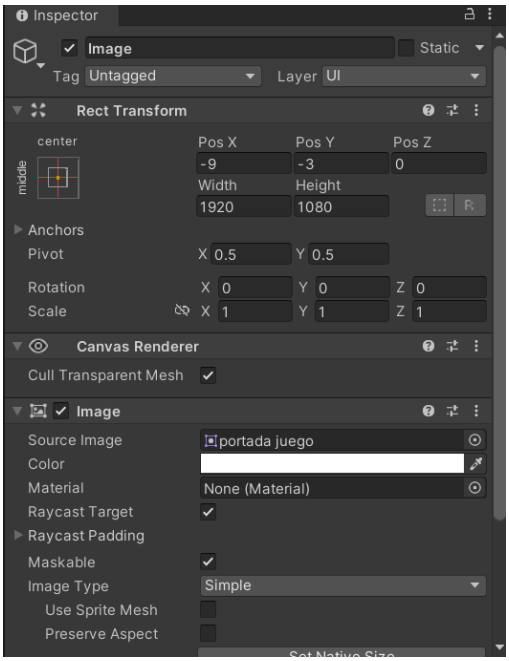




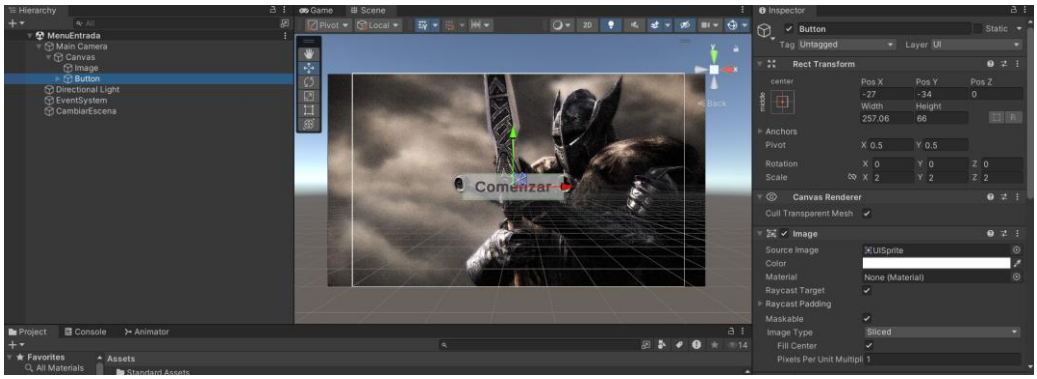
Tras esto, he buscado una imagen para poner de fondo, la he descargado y la he guardado en la carpeta de Assets del proyecto. Para poder usarla, en el inspector de la imagen he cambiado el textureType de Default a Sprite (2D and UI).

Dentro del canvas creamos una Image y le ponemos el Ancho y largo que le corresponde y arrastramos el Sprite donde pone Source Image.

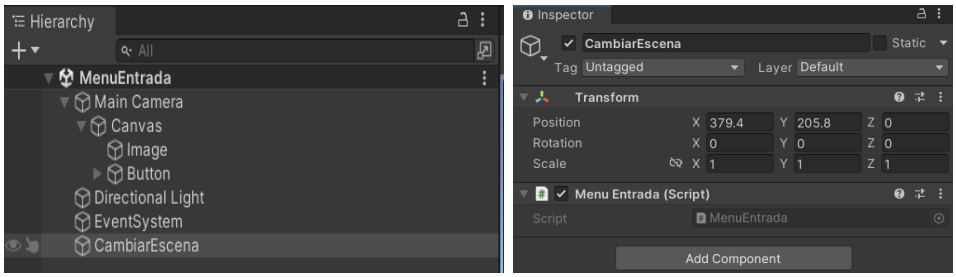




Ya tenemos la imagen de fondo, ahora toca crear el botón y su lógica. Creamos el botón dentro del canvas y le ponemos el texto de comenzar.

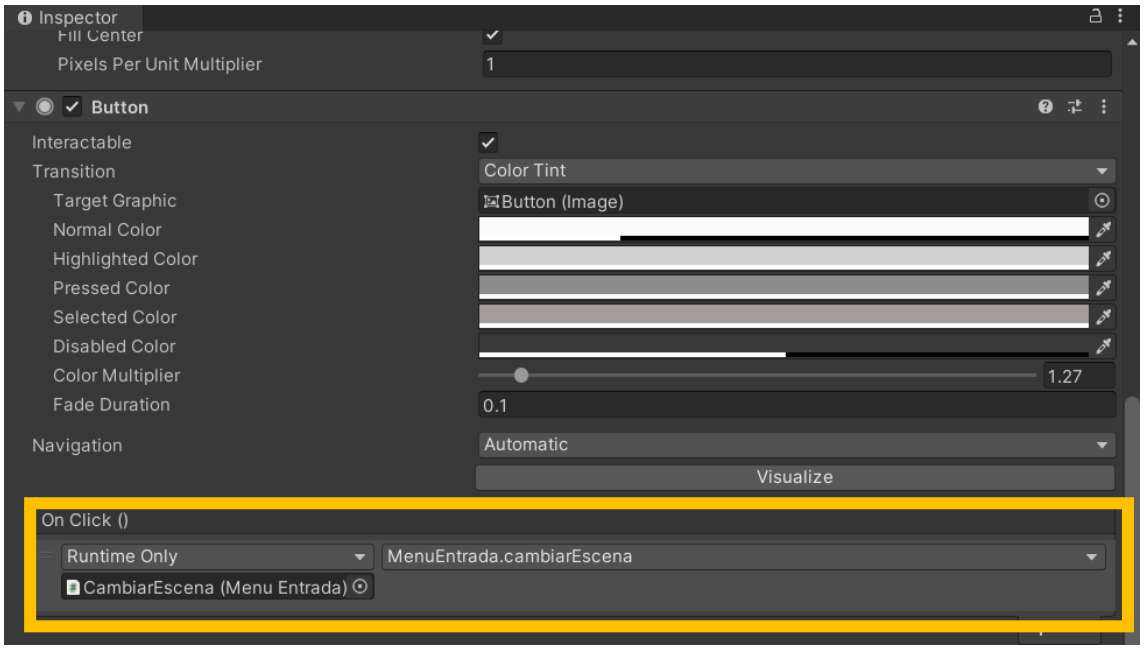


Ahora necesitamos que el botón cambie de escena cuando lo pulsemos. Para esto he creado un GameObject llamado CambiarEscena, el cual, lo único que va a tener es el script MenuEntrada, donde vamos a implementar la función para que cambie de escena.

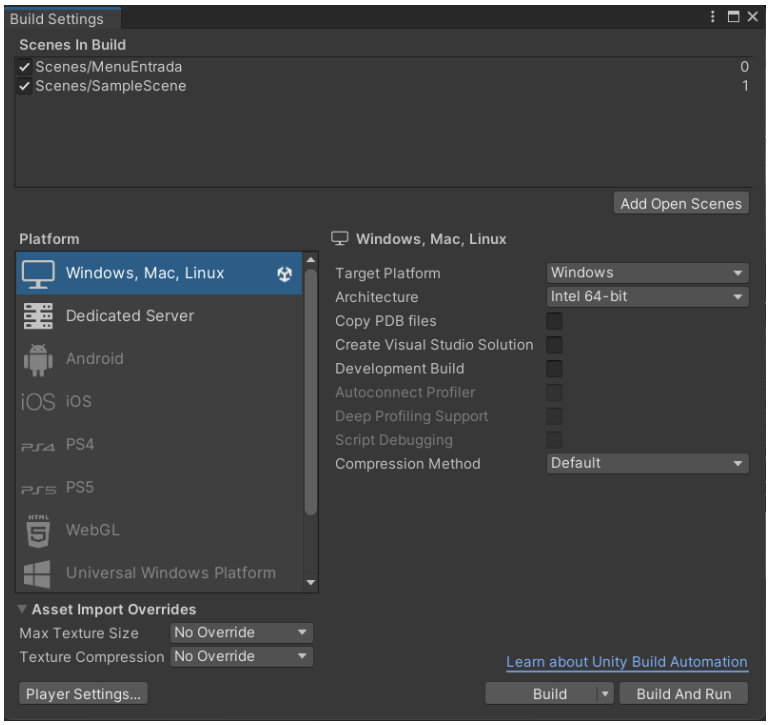


Este script está en la carpeta de scripts y contiene una única función llamada cambiarEscena, la cual lo único que realiza es el cambio de escena a la escena 1 (la siguiente) con un SceneManager.LoadScene(1).

A continuación, en el botón le indico el game object donde está el script al que tiene que acceder, el script, y la función a la que tiene que llamar cuando se pulse el botón.



Ahora hay que establecer las escenas en orden en el build para que cada una corresponda al índice que queremos. Para ello pinchamos en file->Build Settings y añadimos la escena clicando en Add Open Scenes.



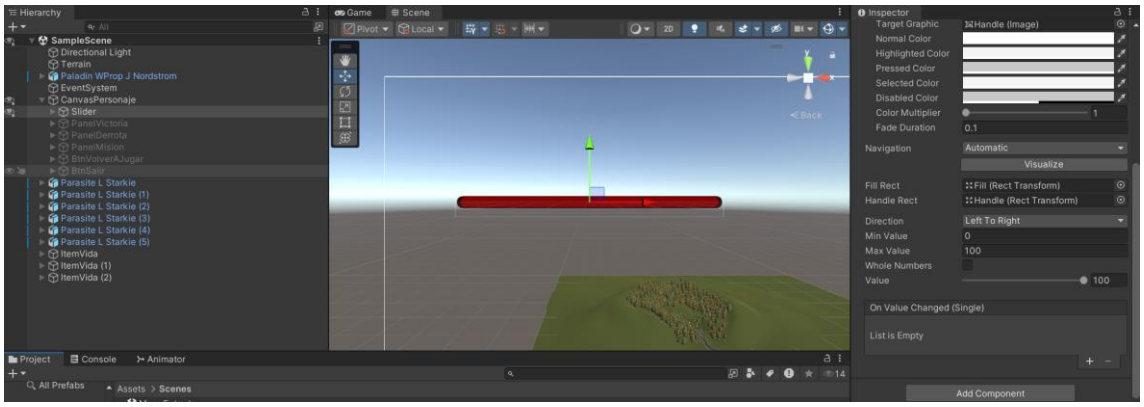
De este modo ya funcionaría el cambio de escena. Al iniciar el juego, clicamos en el botón de comenzar y ese botón llama a la función de cambiarEscena del script MenuEntrada, cargando la escena con índice 1.

CANVAS DEL PERSONAJE Y PANELES DE VICTORIA Y DERROTA

Nota: Este canvas "CanvasPersonaje" va a tener asignado un script llamado ControlCanvas, donde voy a implementar toda la lógica que conlleva.

Slider de vida del personaje y texto con el valor numérico

En la escena de acción, creo un canvas lo llamo CanvasPersonaje y dentro del canvas creo un slider. Cambio el MaxValue del slider a 100 que va a ser la vida total del personaje, y edito el fondo y el fill con colores rojizos.



Ahora, dentro del Slider creamos 2 textos, el primero pone Health, y el segundo el número 100 que más adelante irá cambiando el valor. Edito el color del texto y lo pongo en cursiva.



Toca asignar el valor del slider y el texto para que cada vez que se modifique la vida del jugador, se modifiquen ambos también. Para ello en el script LogicaPaladin asignamos en dos variables sliderVida y numberVida, el slider y el texto correspondiente del canvas y mediante ellas accedemos al valor del slider y al contenido del texto y lo igualamos a la vida del personaje de este modo:

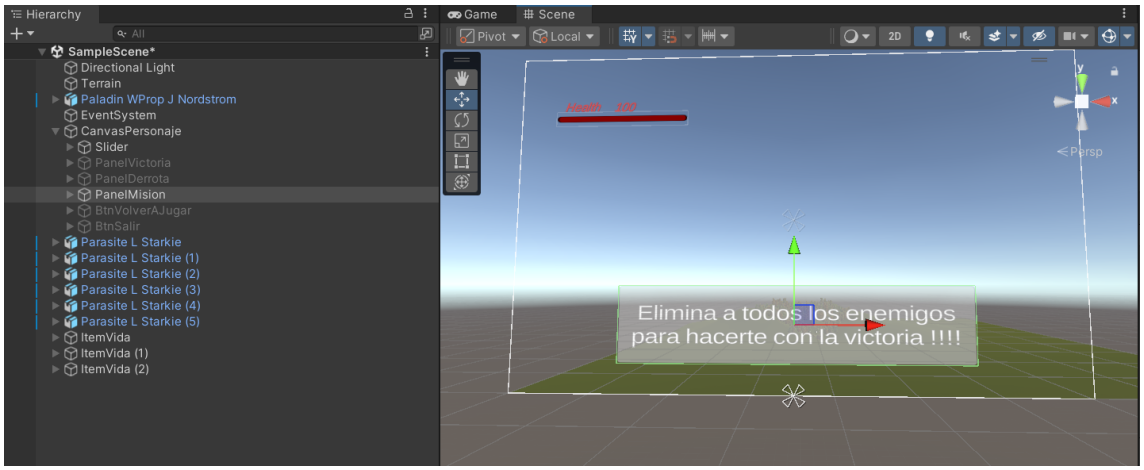
```
void Update()
{
    sliderVida.GetComponent<Slider>().value = vida;
    numberVida.text = vida.ToString();
```

De esta manera, cada vez que cambie la vida del jugador, cambiará el slider y el texto.

Panel con mensaje de misión

Este panel se activará durante 5 segundos al iniciar la escena de acción para que el jugador sepa qué tiene que hacer para ganar.

Dentro de CanvasPersonaje creamos un nuevo panel llamado PanelMision y dentro del panel creamos un el texto en el que escribimos lo que tiene que hacer el jugador. Ajustamos el tamaño del panel a como queremos que se vea en el juego y el color de fondo y texto.



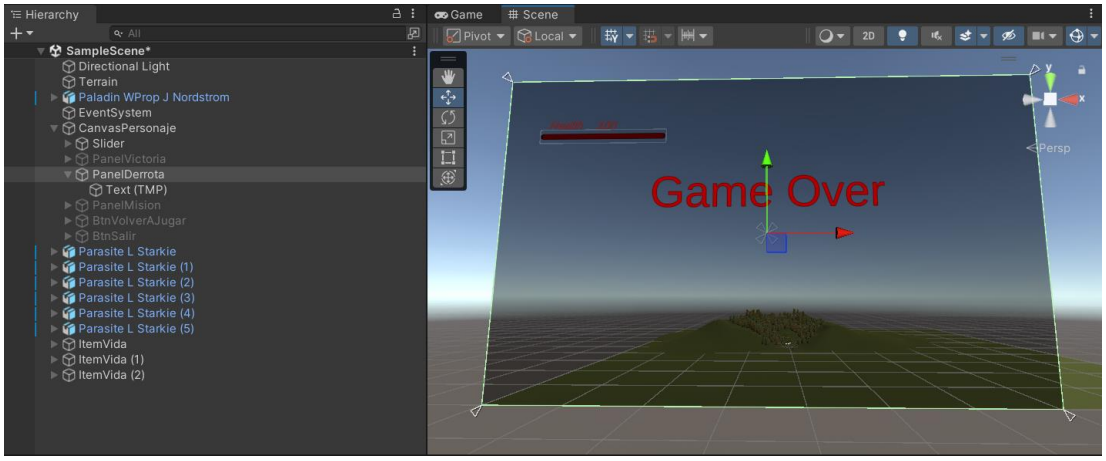
Después dejo el panel desactivado para activarlo y desactivarlo desde el script "ControlCanvas" asignado a CanvasPersonaje. Para esto, asigno el panel PanelMision a una variable pública en el script llamada panelMision, y mediante ésta poder activar o desactivar el panel usando una variable cronómetro. Si el crono está entre 2 y 7 segundos, activamos el panel, el resto del tiempo lo desactivamos.

```
void Update()
{
    crono += Time.deltaTime;
    if((crono > 2f)&&(crono < 7f))
    {
        panelMision.SetActive(true);
    }
    else
    {
        panelMision.SetActive(false);
    }
```

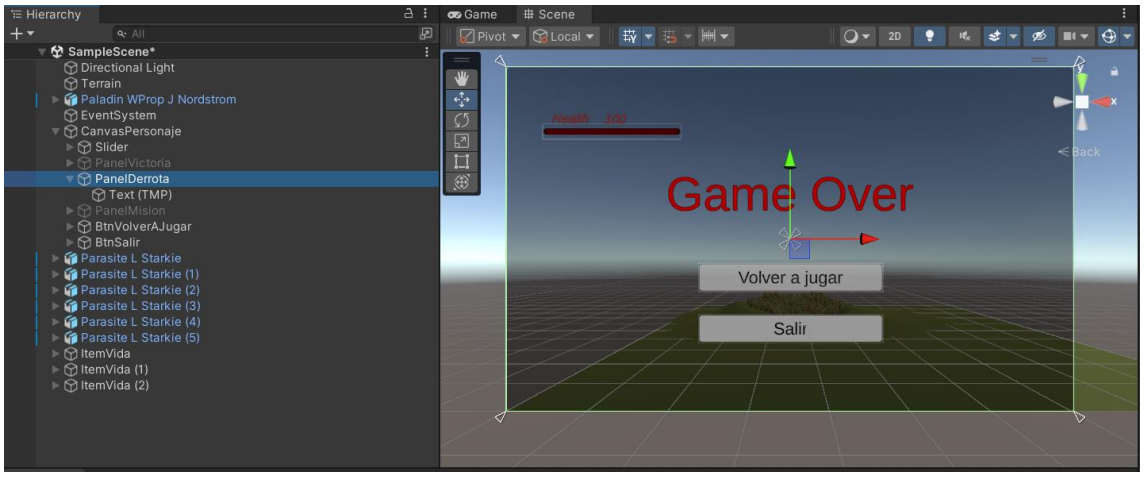
Panel de derrota

Este panel se activará cuando la vida de nuestro personaje sea <= 0. A parte de este panel, crearemos 2 botones para volver a jugar y para salir del juego.

En primer lugar creo el panel dentro de CanvasPersonaje y lo llamo panelDerrota. Dentro de PanelDerrota creo un text y escribo Game Over. Tras eso ajusto el panel y el texto y le edito los colores.



Una vez creado el panel y el texto, creo dos botones dentro de CanvasPersonaje y los llamo BtnVolverAJugar y BtnSalir. Creo los botones y les edito el texto, y los colocamos en pantalla en la disposición que queramos.



Ahora, toca implementar la lógica de dichos botones. En el script ControlCanvas, asociado a CanvasPersonaje, creo dos funciones. La primera se llama volverAJugar, la cual cambia a la escena MenuEntrada, que tiene el índice 0 en el build setting. La segunda función se llama salirDelJuego, la cual hace un Application.Quit.




```
public void volverAJugar()
{
    SceneManager.LoadScene(0);
}
0 referencias
public void salirDelJuego()
{
    Application.Quit();
}
```

Nota: En los OnClick() de los botones pone FinDelJuego. Esto es porque antes el script que ahora es ControlCanvas se llamaba fin del juego, luego le cambié el nombre a ControlCanvas y no se ha actualizado en el OnClick().

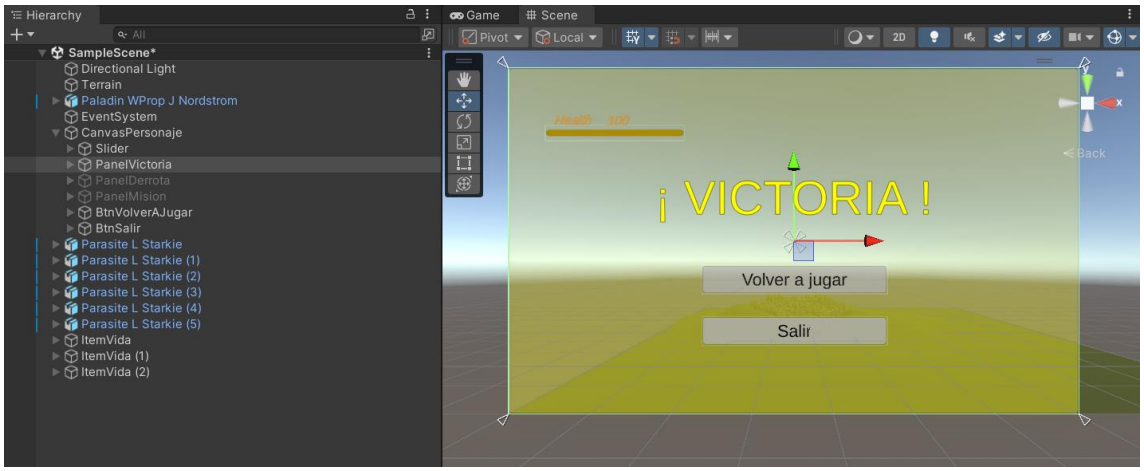
Después de esto, desactivamos los botones y el panel para que sólo se activen cuando el personaje muera. Esto lo hacemos en el script de ControlCanvas de esta manera:

```
if(player.GetComponent<LogicaPaladin>().vida <= 0)
{
    panelDerrota.SetActive(true);
    botonVolverAJugar.SetActive(true);
    botonSalir.SetActive(true);
}
```

Panel de victoria

Para el panel de victoria seguimos los mismos pasos que para el de derrota, solo que le cambiamos el texto y los colores. Como vamos a usar los mismos botones que para el panel de derrota no hace falta que creemos unos nuevos.

El panel finalmente queda así:



Este panel se activa cuando eliminamos a todos los enemigos. Esto lo he implementado en el script ControlCanvas de esta forma:

```
cantidadEnemigos = GameObject.FindGameObjectsWithTag("Enemigo");

if(cantidadEnemigos.Length == 0)
{
    panelVictoria.SetActive(true);
    botonVolverAJugar.SetActive(true);
    botonSalir.SetActive(true);
    player.GetComponent<Animator>().SetBool("Celebration", true);
    player.GetComponent<LogicaPaladin>().enabled = false;

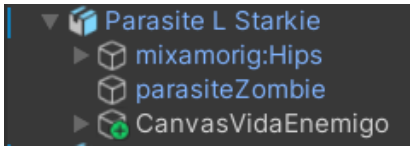
}
```

Donde cantidadEnemigos es un vector de GameObjects que tienen el tag Enemigo.

CANVAS DEL ENEMIGO

La idea es crear un slider de vida como el del personaje del jugador, pero encima de cada enemigo.

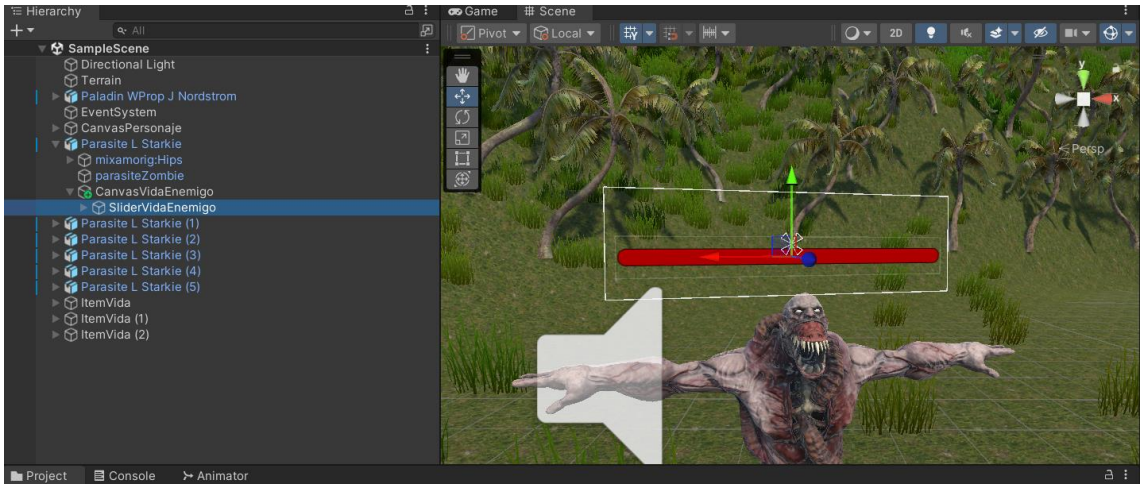
Dentro del GameObject del enemigo creo un canvas para que éste se mueva con el enemigo y lo llamo CanvasVidaEnemigo.



A diferencia del canvas del personaje, éste debe estar en el terreno en vez de la pantalla, concretamente encima de cada enemigo. Para esto, en el canvas marcamos la opción de World Space en el Render Mode, y lo colocamos encima del enemigo.

Slider de vida del enemigo

Una vez creado el canvas y colocado, creo un slider dentro de CanvasVidaEnemigo y lo edito como el del jugador, solo que el del enemigo no tiene el texto numérico.



Ahora es necesario asignar el valor del slider al de la vida del enemigo. Para eso, en el script de LogicaEnemigo he puesto esto par que el valor del slider sea siempre el mismo que el de la vida del enemigo, y a medida que cambie la vida, cambie el valor del slider.

```
void Update()
{
    sliderVida.GetComponent<Slider>().value = vida;
```

sliderVida es una variable de tipo Slider al que la hemos asignado el slider de la vida del enemigo.

Pero como el slider es en 2D, si en el juego nos ponemos de perfil al enemigo no se aprecia. Para solucionar esto he creado un nuevo script llamado CanvasVidaEnemigo y se lo he asigando al canvas del enemigo.

En este script he implementado la lógica para que el canvas rote a mediada que se mueve el jugador para que esté siempre mirando de frente al jugador. He implementado la misma lógica que para que el enemigo rotase hacia la dirección donde estaba el jugador en la persecución. Lo único que ahora en vez de la posición del enemigo, uso la posición de la cámara, para que el slider esté mirando hacia la cámara y no hacia el jugador.

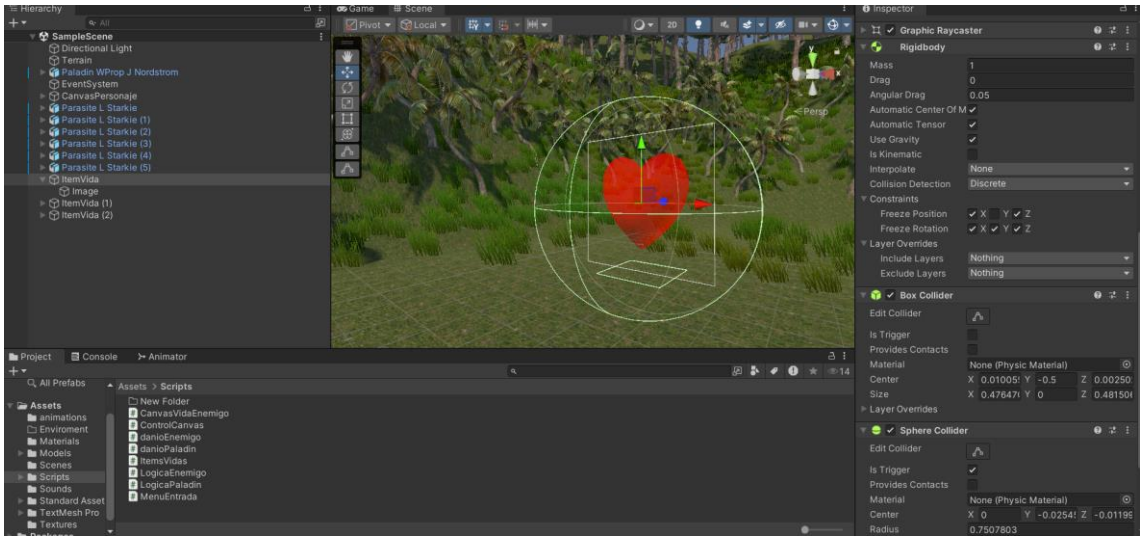
```
18      void Update()
19      {
20          direccion = cameraMain.GetComponent<Transform>().position - transform.position; //obtenemos el vector resul
21          rotation = Quaternion.LookRotation(direccion);  //LookRotation nos proporciona la rotación necesaria para q
22          transform.rotation = Quaternion.RotateTowards(transform.rotation, rotation, VelRotacion * Time.deltaTime);
23      }
24  }
```

ITEMS VIDA

He creado una especie de item el cual, si el personaje del jugador tiene menos de 100 de vida, poder coger el item y aumentar la vida, como si fuese una sanación.

Para el item, he descargado una imagen de un corazón en png y lo he pueso el textura type en Sprite como hice con la imagen del menú de entrada.

Después he creado un nuevo canvas y lo he llamado item vida. Dentro de este canvas he creado una imagen y le he asignado el Sprite del corazón. El canvas debe ser como el del slider de la vida del enemigo, de tipo World Space. Para que el canvas esté en el suelo, le he añadido un rigidBody y un box collider para que se quede encima del suelo. Además le he añadido un sphere collider con la opción de trigger activada para que detecte si el enemigo entra en contacto con el item o no.



Para la lógica del item, he creado un nuevo script llamado itemsVidas que he asignado al canvas ItemVida, el cual va a contener la implementación de la función OnTriggerEnter, y la rotación del canvas para que siempre esté mirando hacia la cámara, exactamente igual que el slider de vida del enemigo.

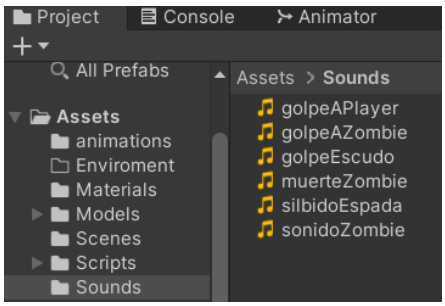
```
    // Update is called once per frame
    Mensaje de Unity | 0 referencias
    void Update()
    {
        direccion = mainCamera.GetComponent<Transform>().position - transform.position; //obtenemos el vector resultante de la resta d
        rotation = Quaternion.LookRotation(direccion);  //LookRotation nos proporciona la rotación necesaria para que el enemigo mire
        transform.rotation = Quaternion.RotateTowards(transform.rotation, rotation, VelRotacion * Time.deltaTime); //aplicamos esa rot
    }

    Mensaje de Unity | 0 referencias
    private void OnTriggerEnter(Collider other)
    {
        if((other.CompareTag("Player")) && (player.GetComponent<LogicaPaladin>().vida < 100))
        {
            if(player.GetComponent<LogicaPaladin>().vida <= 90)
            {
                player.GetComponent<LogicaPaladin>().vida += puntosSalud;
                Destroy(gameObject);
            }
            else
            {
                player.GetComponent<LogicaPaladin>().vida = 100;
                Destroy(gameObject);
            }

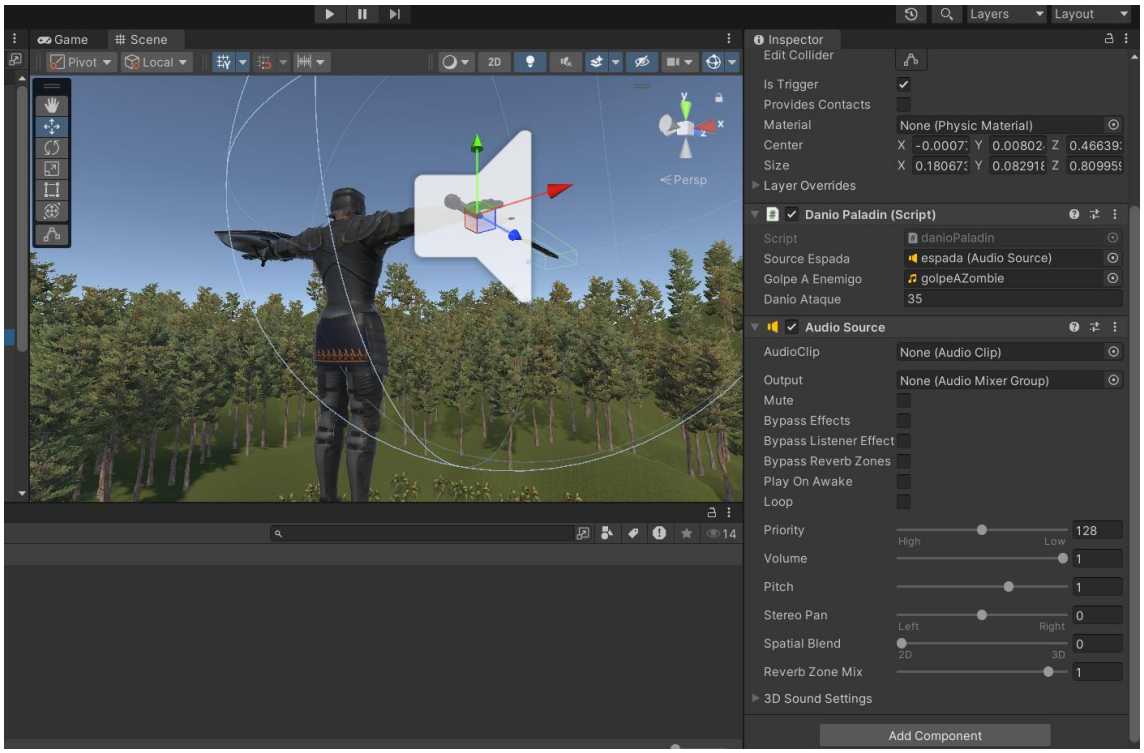

        }
    }
}
```

SONIDOS

He creado una carpeta sounds y he añadido en ella todos los sonidos que he descargado:



Para los sonidos del jugador, he añadido un audio source a la espada, que nos va a servir para reproducir el clip de silbidoEspada cuando ataque, y el cilp golpeAZombie si le golpeamos.



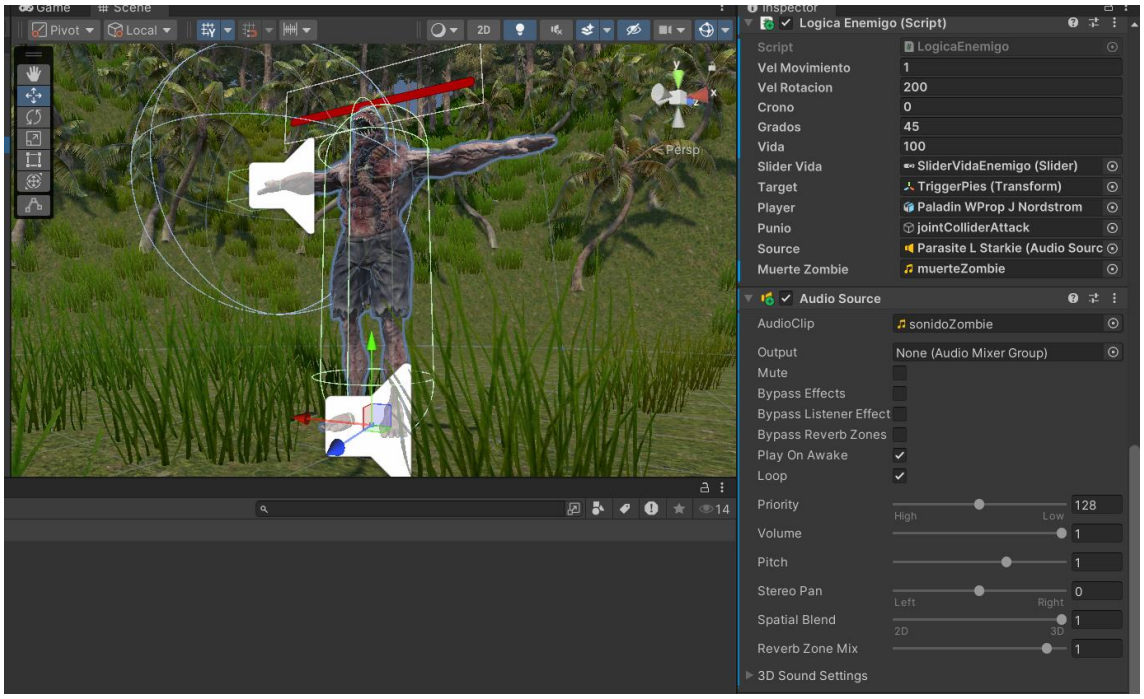
Vemos que en el audio source no le ponemos clip, ya que la intención es usar ese auido source para reproducir los dos clips que he mencionado antes. El audio source y el clip de golpeAZombie se asignan a dos variables que están en el script danioPaladin. El otro clip, silbidoEspada, se asigna a otra variable que está en LogicaPaladin. Esto es así ya que en logicaPaladin se introduce la lógica de ataque, así podemos reproducir el clip en el momento del ataque. El otro clip de golpeAZombie se asigna a daniopaladin ya que ahí se implementa la lógica del trigger de la espada, para saber si impacta en el enemigo o no.

En LogicaPaladin se activa el clip de silbidoEspada cada vez que se ataca, en daniopaladin se activa el clip de golpeAZombie si el trigger detecta que ha colisionado con un enemigo.

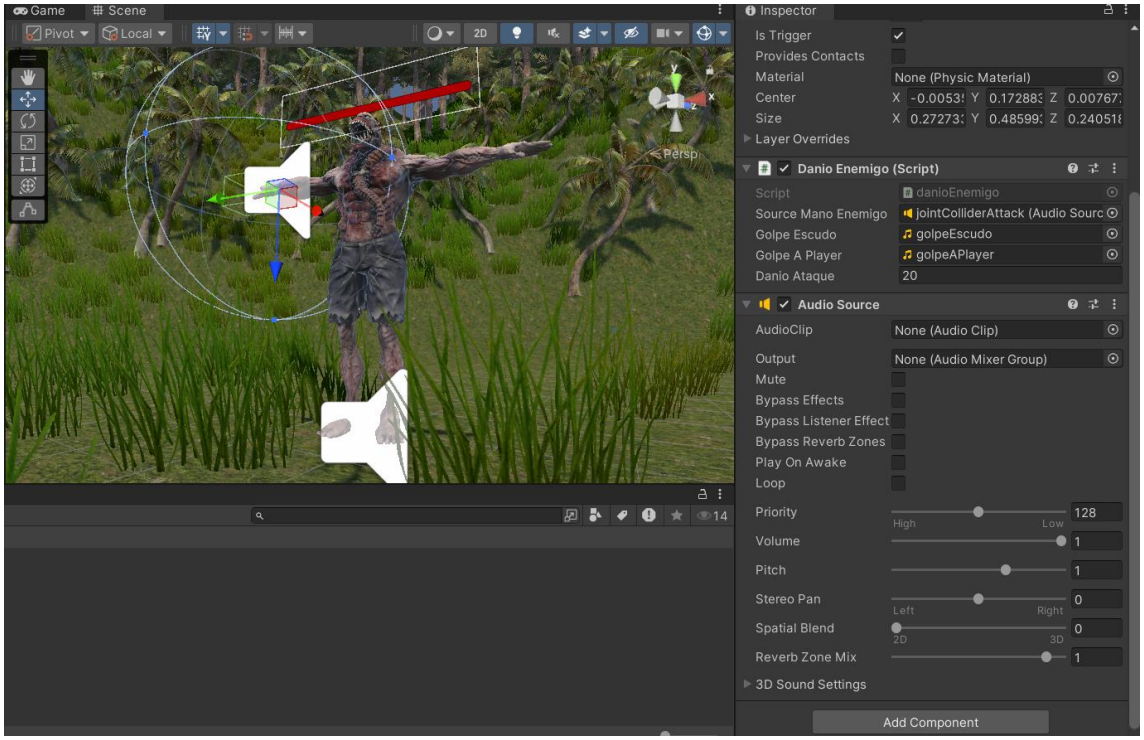
Estos clips van a ser en modo 2D ya que quiero que suenen independientemente de la distancia.

Para los sonidos del enemigo, he creado dos audio source. Uno en el enemigo, y otro en el jointColliderAttack que está en su mano derecha.

El primero de ellos, el del enemigo, va a reproducir el clip de sonidoZombie constantemente, hasta que muera, asique le marco la opción de Loop y Play on Awake para que suene desde el inicio, le edito el alcance para que se deje de escuchar si nos alejamos mucho y asigno dicho clip al audio source directamente. Éste audio source si va a estar en modo 3D y con un Lineal Rolloff.



El segundo audio source, el del jointColliderAttack, no va atener asociado ningún clipa directamente, ya que la intención de este audio source pueda alternar entre el sonido de impacto en el jugador y el sonido de impacto en el escudo. Para ello, en el script danioEnemigo, asociamos el audio source del joint collider, y los dos clips de golpeEscudo y golpeAPlayer.



Este audio source también va a estar en 2D ya que quiero que suene independientemente de la distancia. En el script de danioEnemigo, se reproducirá el

clip del golpeAEscudo si el trigger del puño detecta el escudo, o se reproducirá el clip del golpeAPlayer si el trigger del puño detecta al jugador.

El listener está asociado a la Main camera.